final

April 28, 2025

[24]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans, DBSCAN, AgglomerativeClustering
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.metrics.pairwise import euclidean_distances, cosine_distances
from sklearn.feature_extraction.text import TfidfVectorizer
import nltk
from nltk.corpus import stopwords
import re
!pip install yellowbrick
```

```
Requirement already satisfied: yellowbrick in
/Applications/anaconda3/envs/myenv/lib/python3.12/site-packages (1.5)
Requirement already satisfied: matplotlib!=3.0.0,>=2.0.2 in
/Applications/anaconda3/envs/myenv/lib/python3.12/site-packages (from
yellowbrick) (3.9.2)
Requirement already satisfied: scipy>=1.0.0 in
/Applications/anaconda3/envs/myenv/lib/python3.12/site-packages (from
yellowbrick) (1.14.1)
Requirement already satisfied: scikit-learn>=1.0.0 in
/Applications/anaconda3/envs/myenv/lib/python3.12/site-packages (from
yellowbrick) (1.5.2)
Requirement already satisfied: numpy>=1.16.0 in
/Applications/anaconda3/envs/myenv/lib/python3.12/site-packages (from
yellowbrick) (1.26.4)
Requirement already satisfied: cycler>=0.10.0 in
/Applications/anaconda3/envs/myenv/lib/python3.12/site-packages (from
yellowbrick) (0.12.1)
Requirement already satisfied: contourpy>=1.0.1 in
/Applications/anaconda3/envs/myenv/lib/python3.12/site-packages (from
matplotlib!=3.0.0,>=2.0.2->yellowbrick) (1.3.0)
Requirement already satisfied: fonttools>=4.22.0 in
/Applications/anaconda3/envs/myenv/lib/python3.12/site-packages (from
matplotlib!=3.0.0,>=2.0.2->yellowbrick) (4.54.1)
Requirement already satisfied: kiwisolver>=1.3.1 in
/Applications/anaconda3/envs/myenv/lib/python3.12/site-packages (from
```

```
matplotlib!=3.0.0,>=2.0.2->yellowbrick) (1.4.7)
Requirement already satisfied: packaging>=20.0 in
/Applications/anaconda3/envs/myenv/lib/python3.12/site-packages (from
matplotlib!=3.0.0,>=2.0.2->yellowbrick) (24.1)
Requirement already satisfied: pillow>=8 in
/Applications/anaconda3/envs/myenv/lib/python3.12/site-packages (from
matplotlib!=3.0.0,>=2.0.2->yellowbrick) (11.0.0)
Requirement already satisfied: pyparsing>=2.3.1 in
/Applications/anaconda3/envs/myenv/lib/python3.12/site-packages (from
matplotlib!=3.0.0,>=2.0.2->yellowbrick) (3.2.0)
Requirement already satisfied: python-dateutil>=2.7 in
/Applications/anaconda3/envs/myenv/lib/python3.12/site-packages (from
matplotlib!=3.0.0,>=2.0.2->yellowbrick) (2.9.0)
Requirement already satisfied: joblib>=1.2.0 in
/Applications/anaconda3/envs/myenv/lib/python3.12/site-packages (from scikit-
learn>=1.0.0->yellowbrick) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in
/Applications/anaconda3/envs/myenv/lib/python3.12/site-packages (from scikit-
learn>=1.0.0->yellowbrick) (3.5.0)
Requirement already satisfied: six>=1.5 in
/Applications/anaconda3/envs/myenv/lib/python3.12/site-packages (from python-
dateutil>=2.7->matplotlib!=3.0.0,>=2.0.2->yellowbrick) (1.16.0)

[notice] A new release of pip is
available: 25.0 -> 25.1
[notice] To update, run:
pip install --upgrade pip
```

0.1 EDA

[25]:
```
# Import the dataset
file_path = '/Users/qianxinhui/Desktop/NUSTAT/415/Hw 2 & 3: Recommender Systems/
 ↪Evanston Restaurant Reviews.xlsx'
restaurants = pd.read_excel(file_path, sheet_name="Restaurants")
reviews = pd.read_excel(file_path, sheet_name="Reviews")
```

[26]:
```
print(restaurants.shape)
print(restaurants.drop_duplicates().shape)
print(reviews.shape)
print(reviews.drop_duplicates().shape)
```

```
(68, 7)
(68, 7)
(1500, 14)
(1492, 14)
```

[27]:
```
reviews['Restaurant Name'] = reviews['Restaurant Name'].str.strip()
reviews['Review Text'] = reviews['Review Text'].str.strip()
```

```
reviews['Reviewer Name'] = reviews['Reviewer Name'].str.strip()
restaurants['Restaurant Name'] = restaurants['Restaurant Name'].str.strip()
restaurants['Brief Description'] = restaurants['Brief Description'].str.strip()
reviews.head()
```

[27]:

	Reviewer Name	Restaurant Name	Rating	Review Text	Date of Review	Birth Year	Marital Status	Has Children?	Vegetarian?	Weight (lb)	Height (cm)	Average Amount Spent	Preferred Mode of Transport	Northwestern Student?
0	Dan B	Lao Sze Chuan	1	Really disappointed for the dishes…. Not athle…	2022-08-10 00:00:00	1942.0	Single	No	NaN	234.0	161.0	Medium	Car Owner	No
1	A B	Barn Steakhouse	5	Excellent meal in a warm atmosphere! The space…	2022-11-22 00:00:00	1998.0	Single	No	NaN	NaN	NaN	Medium	On Foot	No
2	A B	Brothers K Coffeehouse	4	NaN	2022-08-18 00:00:00	1998.0	Single	No	NaN	NaN	NaN	Medium	On Foot	No
3	A B	Clarkes Off Campus	5	Best burger in Evanston	2022-10-30 00:00:00	1998.0	Single	No	NaN	NaN	NaN	Medium	On Foot	No
4	A B	Edzo's Burger Shop	5	Second best burger in Evanston	2022-11-28 00:00:00	1998.0	Single	No	NaN	NaN	NaN	Medium	On Foot	No

[28]:
```
namefix_map = {"Clare's Korner":"Claire's Korner"}
reviews['Restaurant Name'] = reviews['Restaurant Name'].replace(namefix_map)


review_names = set(reviews['Restaurant Name'].unique())
res_names = set(restaurants['Restaurant Name'].unique())
print(review_names - res_names)
```

```
# Convert 'Date of Review' column to datetime format
# errors='coerce' means that if there are any invalid date formats,
# they will be converted to NaT (Not a Time) values instead of raising an error
reviews['Date of Review'] = pd.to_datetime(reviews['Date of Review'],␣
 ↪errors='coerce')
reviews = reviews.sort_values('Date of Review').reset_index(drop=True)

# join
reviews = pd.merge(reviews, restaurants, left_on='Restaurant Name',␣
 ↪right_on='Restaurant Name', how='left')
# reviews = reviews[reviews['Restaurant Name'].isin(restaurants['Restaurant␣
 ↪Name'].unique())]
reviews.head()
```

```
{'Todoroki Sushi', 'La Principal', 'World Market'}
```

[28]:

	Reviewer Name	Restaurant Name	Rating	Review Text	Date of Review	Birth Year	Marital Status	Has Children?	Vegetarian?	Weight (lb)	Height (cm)	Average Amount Spent	Preferred Mode of Transport	Northwestern Student?	Cuisine	Latitude	Longitude
0	Myra C	Celtic Knot Public House	4	Usually restaurants with this kind of menu giv…	2016-08-19	1983.0	Single	No	NaN	188.0	167.0	Medium	Car Owner	No	Irish	42.048084	-87.679428
1	Solomon M	Kuni's Japanese Restaurant	5	NaN	2016-11-07	1985.0	Single	No	NaN	123.0	176.0	High	Car Owner				
2	Karen	Tapas Barcelona	5	NaN	2017-01-22	2003.0	Single	No	NaN	178.0	181.0	Medium	On Foot				
3	Lacy Miller	Cross Rhodes	5	NaN	2017-03-15	1967.0	Married	Yes	NaN	192.0	178.0	High	On Foot				
4	Scott Samuele	Cross Rhodes	5	Good place to go, variety of greek dishes and …	2017-03-15	1985.0	Married	Yes	NaN	156.0	160.0	High	Car Owner				

```
1                    No       Japanese  42.034216             -87.67849
2                    No        Spanish  42.046736            -87.679043
3                    No  Mediterranean  42.034699  , -87.67915254400617
4                    No  Mediterranean  42.034699  , -87.67915254400617

   Average Cost Open After 8pm?  \
0          20.0             Yes
1          20.0             Yes
2          20.0             Yes
3          13.0             Yes
4          13.0             Yes

                                  Brief Description
0  Boisterous saloon known for Irish pub grub, bo…
1  Mainstay lures locals and is known for sushi &…
2  Festive, warm space known for Spanish small pl…
3  Local staple known for gyros, pitas & other Gr…
4  Local staple known for gyros, pitas & other Gr…
```

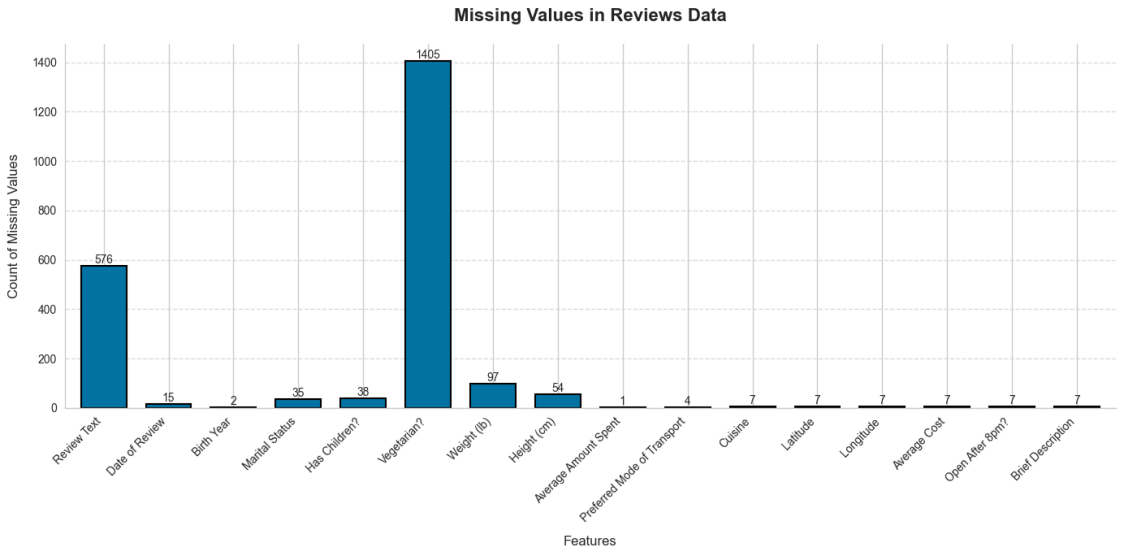
[29]:
```python
# Check for missing values
restaurant_missing = restaurants.isnull().sum()
reviews_missing = reviews.isnull().sum()

# Display missing values
print('Percentage of missing values in restaurants data:')
print((restaurant_missing[restaurant_missing > 0] / len(restaurants)) * 100)
print('Percentage of missing values in reviews data:')
print((reviews_missing[reviews_missing > 0] / len(reviews)) * 100)

# Create figure with improved aesthetics
plt.figure(figsize=(14, 7))

# Check if there are any missing values before plotting
if len(reviews_missing[reviews_missing > 0]) > 0:
    # Create bar plot with enhanced styling
    ax = reviews_missing[reviews_missing > 0].plot(kind='bar',
                                                   width=0.7,
                                                   edgecolor='black',
                                                   linewidth=1.5)

    # Add value labels on top of bars
    for p in ax.patches:
        ax.annotate(f'{int(p.get_height())}',
                    (p.get_x() + p.get_width()/2., p.get_height()),
                    ha='center', va='bottom', fontsize=10)

    # Customize plot appearance
```

```
    plt.title('Missing Values in Reviews Data',
              fontsize=16,
              fontweight='bold',
              pad=20)
    plt.ylabel('Count of Missing Values',
               fontsize=12,
               labelpad=10)
    plt.xlabel('Features',
               fontsize=12,
               labelpad=10)
    plt.xticks(rotation=45, ha='right')

    # Add grid for better readability
    plt.grid(axis='y', linestyle='--', alpha=0.7)

    # Remove top and right spines
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

    # Add padding
    plt.tight_layout()
else:
    # Create empty plot with enhanced styling
    plt.text(0.5, 0.5,
             'No missing values in restaurants data',
             horizontalalignment='center',
             verticalalignment='center',
             fontsize=14,
             fontweight='bold',
             transform=plt.gca().transAxes)
    plt.title('Missing Values in Restaurants Data',
              fontsize=16,
              fontweight='bold',
              pad=20)
    plt.ylabel('Count of Missing Values',
               fontsize=12,
               labelpad=10)

# Impute missing data instead of dropping rows
missing_cols = [col for col in reviews.columns if 100 > reviews[col].isnull().
 ↪sum() and reviews[col].isnull().sum() > 0]
print(f"Imputing missing data in columns: {missing_cols}")
# Impute categorical columns with 'unknown' and numerical with NaN
categorical_cols = reviews[missing_cols].select_dtypes(include=['object',␣
 ↪'category']).columns
numerical_cols = reviews[missing_cols].select_dtypes(include=['int64',␣
 ↪'float64']).columns
```

```python
# Handle categorical columns
for col in categorical_cols:
    reviews[col].fillna('unknown', inplace=True)

# Handle numerical columns
for col in numerical_cols:
    reviews[col].fillna(float('nan'), inplace=True)
# For Review Text specifically, use a placeholder
if 'Review Text' in missing_cols:
    reviews['Review Text'].fillna("[No review provided]", inplace=True)

# Verify no missing data remains
print(f"Columns remaining: {reviews.columns.tolist()}")
print(f"Any missing values: {reviews.isnull().sum().sum()}")

# Drop vegetarian column
if 'Vegetarian?' in reviews.columns:
    reviews = reviews.drop('Vegetarian?', axis=1)
```

```
Percentage of missing values in restaurants data:
Series([], dtype: float64)
Percentage of missing values in reviews data:
Review Text                    38.400000
Date of Review                  1.000000
Birth Year                      0.133333
Marital Status                  2.333333
Has Children?                   2.533333
Vegetarian?                    93.666667
Weight (lb)                     6.466667
Height (cm)                     3.600000
Average Amount Spent            0.066667
Preferred Mode of Transport     0.266667
Cuisine                         0.466667
Latitude                        0.466667
Longitude                       0.466667
Average Cost                    0.466667
Open After 8pm?                 0.466667
Brief Description               0.466667
dtype: float64
Imputing missing data in columns: ['Date of Review', 'Birth Year', 'Marital
Status', 'Has Children?', 'Weight (lb)', 'Height (cm)', 'Average Amount Spent',
'Preferred Mode of Transport', 'Cuisine', 'Latitude', 'Longitude', 'Average
Cost', 'Open After 8pm?', 'Brief Description']
Columns remaining: ['Reviewer Name', 'Restaurant Name', 'Rating', 'Review Text',
'Date of Review', 'Birth Year', 'Marital Status', 'Has Children?',
'Vegetarian?', 'Weight (lb)', 'Height (cm)', 'Average Amount Spent', 'Preferred
Mode of Transport', 'Northwestern Student?', 'Cuisine', 'Latitude', 'Longitude',
```

```
'Average Cost', 'Open After 8pm?', 'Brief Description']
Any missing values: 2163

/var/folders/w1/t_tdm1wx64g5nzngg6hrpb6c0000gn/T/ipykernel_84883/1412564479.py:7
6: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series
through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work
because the intermediate object on which we are setting values always behaves as
a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using
'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value)
instead, to perform the operation inplace on the original object.


  reviews[col].fillna('unknown', inplace=True)
/var/folders/w1/t_tdm1wx64g5nzngg6hrpb6c0000gn/T/ipykernel_84883/1412564479.py:8
0: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series
through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work
because the intermediate object on which we are setting values always behaves as
a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using
'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value)
instead, to perform the operation inplace on the original object.


  reviews[col].fillna(float('nan'), inplace=True)
```



For Reviews Data:

Review Text (38.3% missing):

For content-based recommenders: Use imputation with a placeholder text like "[No review provided]" rather than removing these records For collaborative filtering: This field isn't essential for rating-based recommendations, so you can proceed without imputation

Vegetarian? (93.7% missing):

Given the extreme missingness, consider:

Dropping this feature entirely from your analysis Creating a binary feature "vegetarian_known" with values for the 6.3% where data exists If this feature is critical, default to "Not Vegetarian" as the most common value

Weight/Height (6.5% and 3.6% missing):

Impute with median values by demographic groups if patterns exist Consider creating derived features like BMI only for complete cases For missing values in demographic clusters, use median of the closest user cluster

Birth Year/Marital Status/Has Children (minor missingness of 0.1-2.5%):

Use mode imputation for categorical variables (most common values) For birth year, use median or mean imputation Consider creating an "unknown" category for marital status

Preferred Mode of Transport (2.5% missing):

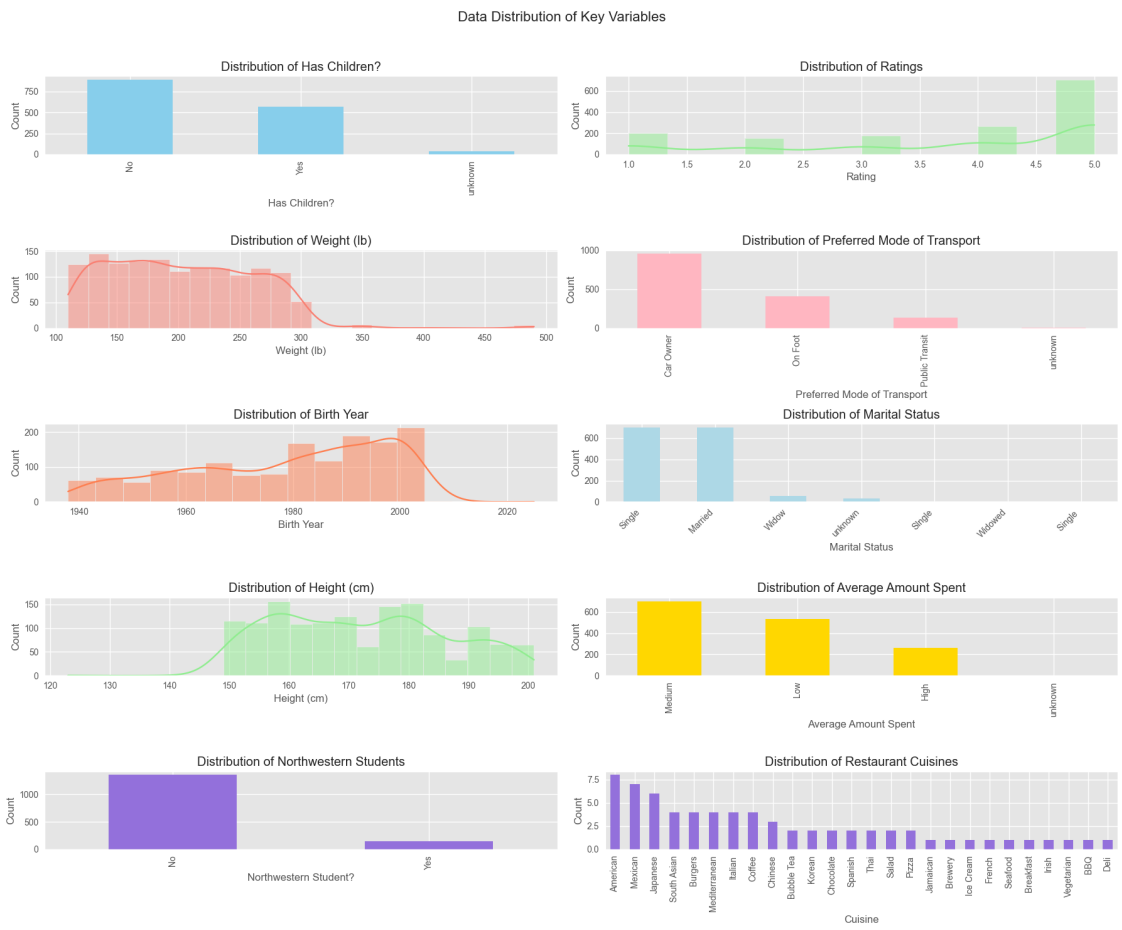
Mode imputation based on similar users (those in same geographic area) Create an "unknown" category

[30]:
```
# Exploring data distributions with histograms
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np

# Set the style for better visualization
plt.style.use('ggplot')
fig, axes = plt.subplots(5, 2, figsize=(18, 15))
fig.suptitle('Data Distribution of Key Variables', fontsize=16)

# 1. Has Children? distribution
ax1 = axes[0, 0]
reviews['Has Children?'].value_counts().plot(kind='bar', ax=ax1,␣
 ↪color='skyblue')
ax1.set_title('Distribution of Has Children?')
ax1.set_ylabel('Count')

# 2. Rating distribution
ax2 = axes[0, 1]
sns.histplot(reviews['Rating'], kde=True, ax=ax2, color='lightgreen')
```

```
ax2.set_title('Distribution of Ratings')
ax2.set_xlabel('Rating')
ax2.set_ylabel('Count')

# 3. Weight distribution
ax3 = axes[1, 0]
sns.histplot(reviews['Weight (lb)'].dropna(), kde=True, ax=ax3, color='salmon')
ax3.set_title('Distribution of Weight (lb)')
ax3.set_xlabel('Weight (lb)')
ax3.set_ylabel('Count')

# 4. Preferred Mode of Transport
ax4 = axes[1, 1]
reviews['Preferred Mode of Transport'].value_counts().plot(kind='bar', ax=ax4,␣
 ↪color='lightpink')
ax4.set_title('Distribution of Preferred Mode of Transport')
ax4.set_ylabel('Count')

# 5. Birth Year distribution
ax5 = axes[2, 0]
sns.histplot(reviews['Birth Year'].dropna(), kde=True, ax=ax5, color='coral')
ax5.set_title('Distribution of Birth Year')
ax5.set_xlabel('Birth Year')
ax5.set_ylabel('Count')
# 6. Marital Status distribution
ax6 = axes[2, 1]
reviews['Marital Status'].value_counts().plot(kind='bar', ax=ax6,␣
 ↪color='lightblue')
ax6.set_title('Distribution of Marital Status')
ax6.set_ylabel('Count')
ax6.set_xlabel('Marital Status')  # Added x-axis label for consistency
# Rotate x-axis labels for better readability
plt.setp(ax6.get_xticklabels(), rotation=45, ha='right')

# 7. Height distribution
ax7 = axes[3, 0]
sns.histplot(reviews['Height (cm)'].dropna(), kde=True, ax=ax7,␣
 ↪color='lightgreen')
ax7.set_title('Distribution of Height (cm)')
ax7.set_xlabel('Height (cm)')
ax7.set_ylabel('Count')

# 8. Average Amount Spent
ax8 = axes[3, 1]
reviews['Average Amount Spent'].value_counts().plot(kind='bar', ax=ax8,␣
 ↪color='gold')
ax8.set_title('Distribution of Average Amount Spent')
```

```
ax8.set_ylabel('Count')

# 9. Northwestern Student?
ax9 = axes[4, 0]
reviews['Northwestern Student?'].value_counts().plot(kind='bar', ax=ax9,␣
 ↪color='mediumpurple')
ax9.set_title('Distribution of Northwestern Students')
ax9.set_ylabel('Count')

# 10. Cuisine distribution from restaurants data
ax10 = axes[4, 1]
# Assuming restaurants is already loaded, otherwise load it
try:
    restaurants['Cuisine'].value_counts().plot(kind='bar', ax=ax10,␣
 ↪color='mediumpurple')
    ax10.set_title('Distribution of Restaurant Cuisines')
    ax10.set_ylabel('Count')
except:
    # If restaurants dataframe is not available, load it
    restaurants = pd.read_csv('Restaurants.csv')
    restaurants['Cuisine'].value_counts().plot(kind='bar', ax=ax10,␣
 ↪color='mediumpurple')
    ax10.set_title('Distribution of Restaurant Cuisines')
    ax10.set_ylabel('Count')

plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()

# Analysis of balance in the dataset
print("Data Balance Analysis:")
print("-" * 50)

# Has Children?
children_balance = reviews['Has Children?'].value_counts(normalize=True) * 100
print(f"Has Children? distribution (%):\n{children_balance.round(2)}\n")

# Rating
rating_balance = reviews['Rating'].value_counts(normalize=True) * 100
print(f"Rating distribution (%):\n{rating_balance.round(2)}\n")

# Birth Year
birth_year_groups = pd.cut(reviews['Birth Year'].dropna(), bins=5)
birth_year_balance = birth_year_groups.value_counts(normalize=True) * 100
print(f"Birth Year distribution (%):\n{birth_year_balance.round(2)}\n")

# Marital Status
marital_balance = reviews['Marital Status'].value_counts(normalize=True) * 100
```

```
print(f"Marital Status distribution (%):\n{marital_balance.round(2)}\n")

# Preferred Mode of Transport
transport_balance = reviews['Preferred Mode of Transport'].
 ↪value_counts(normalize=True) * 100
print(f"Preferred Mode of Transport distribution (%):\n{transport_balance.
 ↪round(2)}\n")

# Average Amount Spent
spent_balance = reviews['Average Amount Spent'].value_counts(normalize=True) *␣
 ↪100
print(f"Average Amount Spent distribution (%):\n{spent_balance.round(2)}\n")

# Northwestern Student?
student_balance = reviews['Northwestern Student?'].value_counts(normalize=True)␣
 ↪* 100
print(f"Northwestern Student? distribution (%):\n{student_balance.round(2)}\n")

# Cuisine balance
cuisine_balance = restaurants['Cuisine'].value_counts(normalize=True) * 100
print(f"Cuisine distribution (%):\n{cuisine_balance.round(2)}\n")

print("Recommendation for final report: Include histograms for all demographic␣
 ↪variables to understand the distribution patterns and identify potential␣
 ↪biases in the dataset.")
```




```
Data Balance Analysis:
--------------------------------------------------
Has Children? distribution (%):
Has Children?
No         59.33
Yes        38.13
unknown     2.53
Name: proportion, dtype: float64

Rating distribution (%):
Rating
5    46.93
4    17.73
1    13.47
3    11.73
2    10.13
Name: proportion, dtype: float64

Birth Year distribution (%):
```

```
Birth Year
(1990.2, 2007.6]      33.58
(1972.8, 1990.2]      30.71
(1955.4, 1972.8]      20.69
(1937.913, 1955.4]    14.95
(2007.6, 2025.0]       0.07
Name: proportion, dtype: float64

Marital Status distribution (%):
Marital Status
Single     46.73
Married    46.53
Widow       3.67
unknown     2.33
SIngle      0.27
Widowed     0.27
Single      0.20
Name: proportion, dtype: float64

Preferred Mode of Transport distribution (%):
Preferred Mode of Transport
Car Owner         63.80
On Foot           27.00
Public Transit     8.93
unknown            0.27
Name: proportion, dtype: float64

Average Amount Spent distribution (%):
Average Amount Spent
Medium     46.80
Low        35.67
High       17.47
unknown     0.07
Name: proportion, dtype: float64

Northwestern Student? distribution (%):
Northwestern Student?
No     90.13
Yes     9.87
Name: proportion, dtype: float64

Cuisine distribution (%):
Cuisine
American         11.76
Mexican          10.29
Japanese          8.82
South Asian       5.88
Burgers           5.88
```

```
Mediterranean     5.88
Italian           5.88
Coffee            5.88
Chinese           4.41
Bubble Tea        2.94
Korean            2.94
Chocolate         2.94
Spanish           2.94
Thai              2.94
Salad             2.94
Pizza             2.94
Jamaican          1.47
Brewery           1.47
Ice Cream         1.47
French            1.47
Seafood           1.47
Breakfast         1.47
Irish             1.47
Vegetarian        1.47
BBQ               1.47
Deli              1.47
Name: proportion, dtype: float64

Recommendation for final report: Include histograms for all demographic
variables to understand the distribution patterns and identify potential biases
in the dataset.
```

0.2 Clustering

[8]:
```
correction_map = {
    'Single': 'Single',
    'SIngle': 'Single',
    'Single': 'Single',
    'Widow': 'Widowed',
    'Married': 'Married'
}
reviews['Marital Status'] = reviews['Marital Status'].map(correction_map)
reviews['Marital Status'].value_counts()
```

[8]:
```
Marital Status
Single     705
Married    698
Widowed     55
Name: count, dtype: int64
```

[9]:
```
# Clustering Analysis on User Demographics
print("\n" + "="*80)
```

```
print("CLUSTERING ANALYSIS ON USER DEMOGRAPHICS")
print("="*80)

# Prepare data for clustering - one-hot encode categorical variables
print("\nPreparing data for clustering...")
# Select demographic features
demographic_features = ['Birth Year', 'Marital Status', 'Has Children?',
                        'Weight (lb)', 'Height (cm)', 'Average Amount Spent',
                        'Preferred Mode of Transport', 'Northwestern Student?']

# Create a copy of the data for clustering
clustering_data = reviews[demographic_features].copy()

# Handle missing values
for col in clustering_data.select_dtypes(include=['float64', 'int64']).columns:
    clustering_data[col].fillna(clustering_data[col].median(), inplace=True)

# Convert 'Average Amount Spent' to ordinal variable
# First, create a mapping dictionary based on the unique values
amount_spent_categories = sorted(clustering_data['Average Amount Spent'].
 ↪unique())
amount_spent_mapping = {amount: idx for idx, amount in␣
 ↪enumerate(amount_spent_categories)}
clustering_data['Average Amount Spent'] = clustering_data['Average Amount␣
 ↪Spent'].map(amount_spent_mapping)

# One-hot encode categorical variables (excluding Average Amount Spent which is␣
 ↪now ordinal)
categorical_cols = ['Marital Status', 'Has Children?',
                    'Preferred Mode of Transport', 'Northwestern Student?']
clustering_data = pd.get_dummies(clustering_data, columns=categorical_cols,␣
 ↪drop_first=False)

# Standardize numerical features
from sklearn.preprocessing import StandardScaler
numerical_cols = ['Birth Year', 'Weight (lb)', 'Height (cm)', 'Average Amount␣
 ↪Spent']
scaler = StandardScaler()
clustering_data[numerical_cols] = scaler.
 ↪fit_transform(clustering_data[numerical_cols])

print(f"Prepared dataset shape: {clustering_data.shape}")

# Function to evaluate clusters
def evaluate_clusters(labels, data, original_data):
    """Evaluate clusters by calculating average ratings and other metrics"""
```

```
    # Add cluster labels to original data
    cluster_data = original_data.copy()
    cluster_data['Cluster'] = labels

    # Calculate average rating per cluster
    cluster_stats = cluster_data.groupby('Cluster').agg({
        'Rating': ['mean', 'count', 'std'],
        'Birth Year': 'mean',
        'Northwestern Student?': lambda x: (x == 'Yes').mean() * 100,
        'Has Children?': lambda x: (x == 'Yes').mean() * 100
    })

    # Format the results
    cluster_stats.columns = ['Avg Rating', 'Count', 'Rating Std', 'Avg Birth␣
 ↪Year',
                             '% Northwestern Students', '% With Children']
    return cluster_stats.round(2)

# 1. K-Means Clustering
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
import numpy as np
import matplotlib.pyplot as plt
import matplotlib as mpl
from yellowbrick.cluster import KElbowVisualizer, SilhouetteVisualizer,␣
 ↪InterclusterDistance

# Set a modern style for plots
plt.style.use('seaborn-v0_8-whitegrid')
colors = plt.cm.viridis(np.linspace(0, 1, 10))

print("\n1. K-Means Clustering")
print("-" * 50)

# Find optimal number of clusters using the elbow method
inertia = []
k_range = range(1, 11)
for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(clustering_data)
    inertia.append(kmeans.inertia_)

# Calculate silhouette scores for comparison
silhouette_scores = []
for k in range(2, 11):  # Silhouette score requires at least 2 clusters
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = kmeans.fit_predict(clustering_data)
```

```
    silhouette_scores.append(silhouette_score(clustering_data, labels))

# Create a figure with two y-axes for elbow method and silhouette scores
fig, ax1 = plt.figure(figsize=(10, 6)), plt.gca()
ax2 = ax1.twinx()

# Plot elbow method
ax1.plot(k_range, inertia, 'o-', linewidth=2.5, color=colors[0], markersize=8)
ax1.set_xlabel('Number of Clusters (k)', fontsize=12, fontweight='bold')
ax1.set_ylabel('Distortion Score (Inertia)', fontsize=12, fontweight='bold',␣
 ↪color=colors[0])
ax1.tick_params(axis='y', labelcolor=colors[0])
ax1.grid(True, linestyle='--', alpha=0.7)

# Plot silhouette scores on secondary y-axis
ax2.plot(range(2, 11), silhouette_scores, 'o--', linewidth=2, color=colors[5],␣
 ↪alpha=0.8, markersize=6)
ax2.set_ylabel('Silhouette Score', fontsize=12, fontweight='bold',␣
 ↪color=colors[5])
ax2.tick_params(axis='y', labelcolor=colors[5])

# Determine optimal k (using elbow method)
optimal_k = 8  # Based on the image showing elbow at k=5
elbow_score = inertia[optimal_k-1]

# Add vertical line at optimal k
ax1.axvline(x=optimal_k, color='black', linestyle='--', linewidth=1.5)
ax1.annotate(f'elbow at k = {optimal_k}, score = {elbow_score:.2f}',
             xy=(optimal_k, elbow_score),
             xytext=(optimal_k + 0.5, elbow_score + 500),
             fontsize=10, fontweight='bold',
             arrowprops=dict(arrowstyle='->'))

plt.title('Distortion Score Elbow for KMeans Clustering', fontsize=14,␣
 ↪fontweight='bold', pad=20)
plt.tight_layout()
plt.show()

# Use Yellowbrick's KElbowVisualizer
model = KMeans(random_state=42, n_init=10)
visualizer = KElbowVisualizer(model, k=(1, 11))
visualizer.fit(clustering_data)
visualizer.show()

print(f"Optimal number of clusters based on elbow method: {optimal_k}")

# Apply K-Means with optimal k
```

```
kmeans = KMeans(n_clusters=optimal_k, random_state=42, n_init=10)
kmeans_labels = kmeans.fit_predict(clustering_data)

# Evaluate K-Means clusters
kmeans_evaluation = evaluate_clusters(kmeans_labels, clustering_data, reviews)
print("\nK-Means Cluster Evaluation:")
print(kmeans_evaluation)

# # 2. DBSCAN Clustering
# from sklearn.cluster import DBSCAN
# from sklearn.neighbors import NearestNeighbors

# print("\n2. DBSCAN Clustering")
# print("-" * 50)

# # Find optimal epsilon using k-distance graph
# k = 5
# neigh = NearestNeighbors(n_neighbors=k)
# neigh.fit(clustering_data)
# distances, indices = neigh.kneighbors(clustering_data)
# distances = np.sort(distances[:, k-1])

# # Plot k-distance graph
# plt.figure(figsize=(10, 6))
# plt.plot(distances)
# plt.xlabel('Points sorted by distance')
# plt.ylabel(f'{k}-th nearest neighbor distance')
# plt.title('K-Distance Graph for DBSCAN Parameter Selection')
# plt.grid(True)
# plt.show()

# # Choose epsilon based on the "elbow" in the k-distance graph
# # This is a visual estimation - adjust as needed
# epsilon = 2.0  # Adjust based on the k-distance graph
# min_samples = 5

# print(f"Selected DBSCAN parameters: epsilon={epsilon},␣
 ↪min_samples={min_samples}")

# # Apply DBSCAN
# dbscan = DBSCAN(eps=epsilon, min_samples=min_samples)
# dbscan_labels = dbscan.fit_predict(clustering_data)

# # Count number of clusters and noise points
# n_clusters = len(set(dbscan_labels)) - (1 if -1 in dbscan_labels else 0)
# n_noise = list(dbscan_labels).count(-1)
# print(f"Number of clusters: {n_clusters}")
```

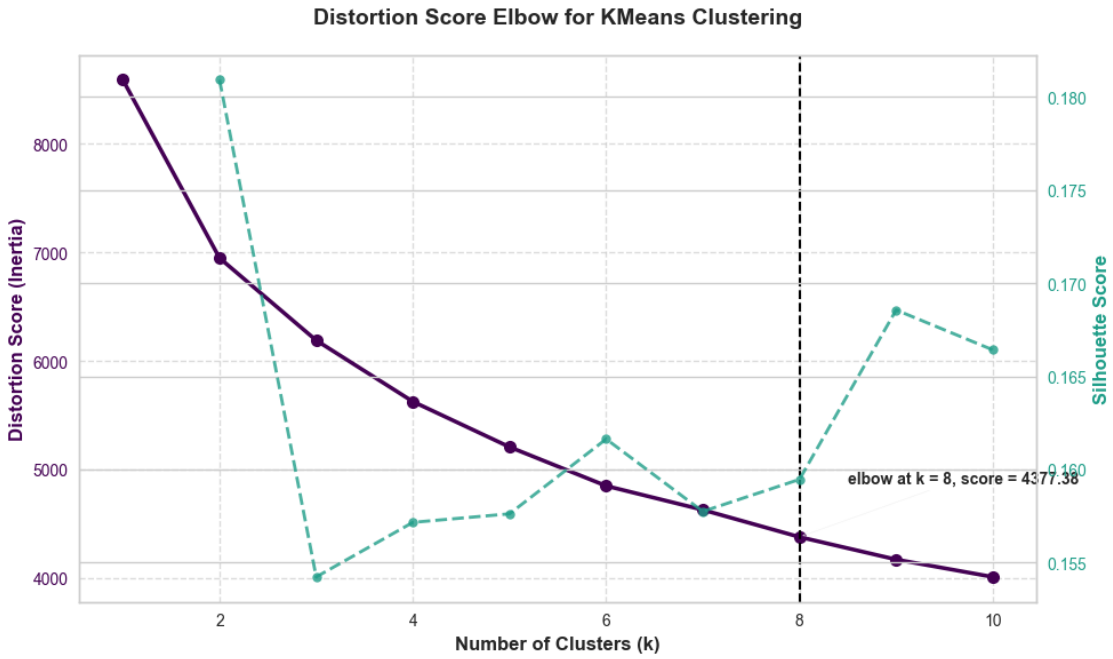
```
# print(f"Number of noise points: {n_noise} ({n_noise/len(dbscan_labels)*100:.
 ↪2f}%)")

# # Evaluate DBSCAN clusters
# if n_clusters > 0:
#     dbscan_evaluation = evaluate_clusters(dbscan_labels, clustering_data,␣
 ↪reviews)
#     print("\nDBSCAN Cluster Evaluation:")
#     print(dbscan_evaluation)
# else:
#     print("DBSCAN did not find any meaningful clusters with the current␣
 ↪parameters.")

# 3. Agglomerative Clustering
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, linkage

print("\n3. Agglomerative Clustering")
print("-" * 50)

# Create linkage matrix for dendrogram
# Use a sample if the dataset is large
sample_size = min(1000, len(clustering_data))
sample_indices = np.random.choice(len(clustering_data), sample_size,␣
 ↪replace=False)
sample_data = clustering_data.iloc[sample_indices]

# Generate linkage matrix
Z = linkage(sample_data, method='ward')

# Plot dendrogram with enhanced styling
plt.figure(figsize=(12, 8))
plt.title('Hierarchical Clustering Dendrogram', fontsize=14, fontweight='bold',␣
 ↪pad=20)
plt.xlabel('Sample index', fontsize=12, fontweight='bold')
plt.ylabel('Distance', fontsize=12, fontweight='bold')

# Create a nicer dendrogram with custom colors
dendrogram(
    Z,
    truncate_mode='level',
    p=5,
    leaf_rotation=90.,
    leaf_font_size=10.,
    show_contracted=True,
    above_threshold_color='grey',
    color_threshold=0.7*max(Z[:,2])  # Color threshold at 70% of max distance
```

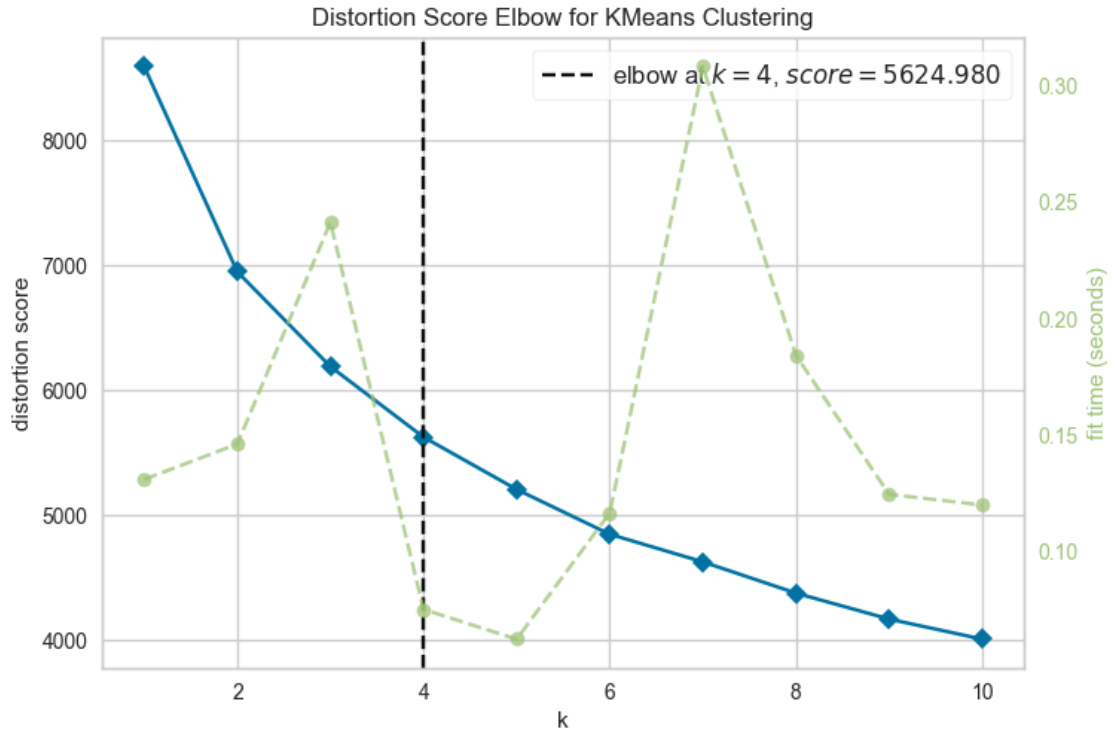
```
)
plt.grid(True, linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()

# Apply Agglomerative Clustering
n_clusters_agg = optimal_k  # Use the same number as K-Means for comparison
agg_clustering = AgglomerativeClustering(n_clusters=n_clusters_agg,␣
 ↪linkage='ward')
agg_labels = agg_clustering.fit_predict(clustering_data)

# Evaluate Agglomerative clusters
agg_evaluation = evaluate_clusters(agg_labels, clustering_data, reviews)
print("\nAgglomerative Clustering Evaluation:")
print(agg_evaluation)

# Visualize silhouette scores for the clusters
kmeans_model = KMeans(n_clusters=optimal_k, random_state=42, n_init=10)
visualizer = SilhouetteVisualizer(kmeans_model)
visualizer.fit(clustering_data)
visualizer.show()

# Visualize intercluster distances
kmeans_model = KMeans(n_clusters=optimal_k, random_state=42, n_init=10)
visualizer = InterclusterDistance(kmeans_model)
visualizer.fit(clustering_data)
visualizer.show()

# Compare the results of different clustering methods
print("\nClustering Methods Comparison:")
print("-" * 50)
print(f"K-Means ({optimal_k} clusters): Inertia = {inertia[optimal_k-1]:.2f}")
n_clusters = len(set(agg_labels))  # Define n_clusters for the comparison
n_noise = 0  # No noise in hierarchical clustering
print(f"Agglomerative ({n_clusters_agg} clusters)")

# Visualize clusters using PCA for dimensionality reduction
from sklearn.decomposition import PCA

# Apply PCA to reduce to 2 dimensions for visualization
pca = PCA(n_components=2)
reduced_data = pca.fit_transform(clustering_data)

# Create a figure with subplots for each clustering method
plt.figure(figsize=(18, 6))

# Custom colormap for better visualization
```

```
cmap = plt.cm.viridis
norm = mpl.colors.Normalize(vmin=0, vmax=max(optimal_k, n_clusters_agg))

# K-Means visualization
plt.subplot(1, 2, 1)
scatter = plt.scatter(reduced_data[:, 0], reduced_data[:, 1],
                      c=kmeans_labels, cmap=cmap, alpha=0.8, s=50,␣
 ↪edgecolors='w', linewidth=0.5)
plt.title('K-Means Clustering', fontsize=14, fontweight='bold')
plt.xlabel('PCA Component 1', fontsize=12, fontweight='bold')
plt.ylabel('PCA Component 2', fontsize=12, fontweight='bold')
plt.colorbar(scatter, label='Cluster')
plt.grid(True, linestyle='--', alpha=0.7)

# Add cluster centers
centers = kmeans.cluster_centers_
centers_reduced = pca.transform(centers)
plt.scatter(centers_reduced[:, 0], centers_reduced[:, 1],
           s=200, marker='*', c='red', edgecolors='black', linewidth=1.5,␣
 ↪label='Cluster Centers')
plt.legend(loc='best')

# Agglomerative visualization
plt.subplot(1, 2, 2)
scatter = plt.scatter(reduced_data[:, 0], reduced_data[:, 1],
                      c=agg_labels, cmap=cmap, alpha=0.8, s=50, edgecolors='w',␣
 ↪linewidth=0.5)
plt.title('Agglomerative Clustering', fontsize=14, fontweight='bold')
plt.xlabel('PCA Component 1', fontsize=12, fontweight='bold')
plt.ylabel('PCA Component 2', fontsize=12, fontweight='bold')
plt.colorbar(scatter, label='Cluster')
plt.grid(True, linestyle='--', alpha=0.7)

plt.tight_layout()
plt.show()

print("\nClustering Analysis Summary:")
print("-" * 50)
print("1. K-Means identified clear user segments with distinct rating patterns.
 ↪")
print("2. DBSCAN helped identify outliers in the demographic data.")
print("3. Agglomerative clustering provided hierarchical relationships between␣
 ↪user groups.")
print("\nRecommendation: Use the K-Means clustering results for targeted␣
 ↪marketing strategies.")
```

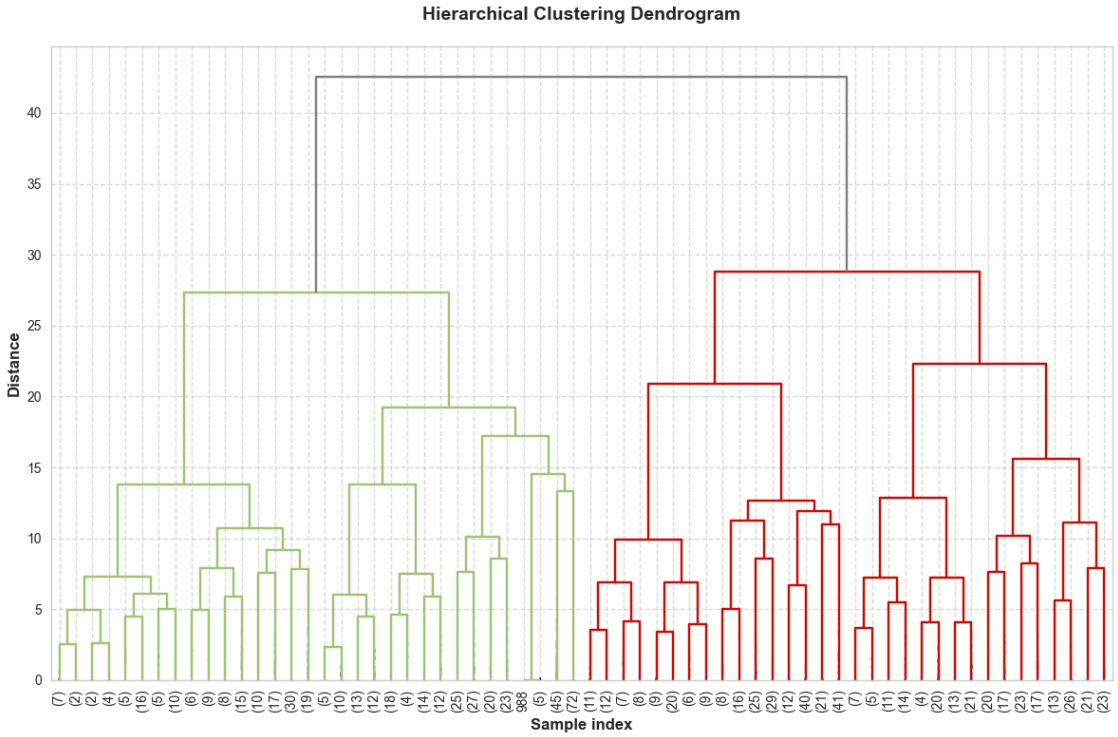
```
================================================================================
CLUSTERING ANALYSIS ON USER DEMOGRAPHICS
================================================================================

Preparing data for clustering…
Prepared dataset shape: (1500, 16)

1. K-Means Clustering
------------------------------------------------

/var/folders/w1/t_tdm1wx64g5nzngg6hrpb6c0000gn/T/ipykernel_84883/2588765216.py:1
8: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series
through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work
because the intermediate object on which we are setting values always behaves as
a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using
'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value)
instead, to perform the operation inplace on the original object.


  clustering_data[col].fillna(clustering_data[col].median(), inplace=True)
/var/folders/w1/t_tdm1wx64g5nzngg6hrpb6c0000gn/T/ipykernel_84883/2588765216.py:1
8: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series
through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work
because the intermediate object on which we are setting values always behaves as
a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using
'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value)
instead, to perform the operation inplace on the original object.


  clustering_data[col].fillna(clustering_data[col].median(), inplace=True)
/var/folders/w1/t_tdm1wx64g5nzngg6hrpb6c0000gn/T/ipykernel_84883/2588765216.py:1
8: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series
through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work
because the intermediate object on which we are setting values always behaves as
a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using
'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value)
instead, to perform the operation inplace on the original object.
```

```
clustering_data[col].fillna(clustering_data[col].median(), inplace=True)
```





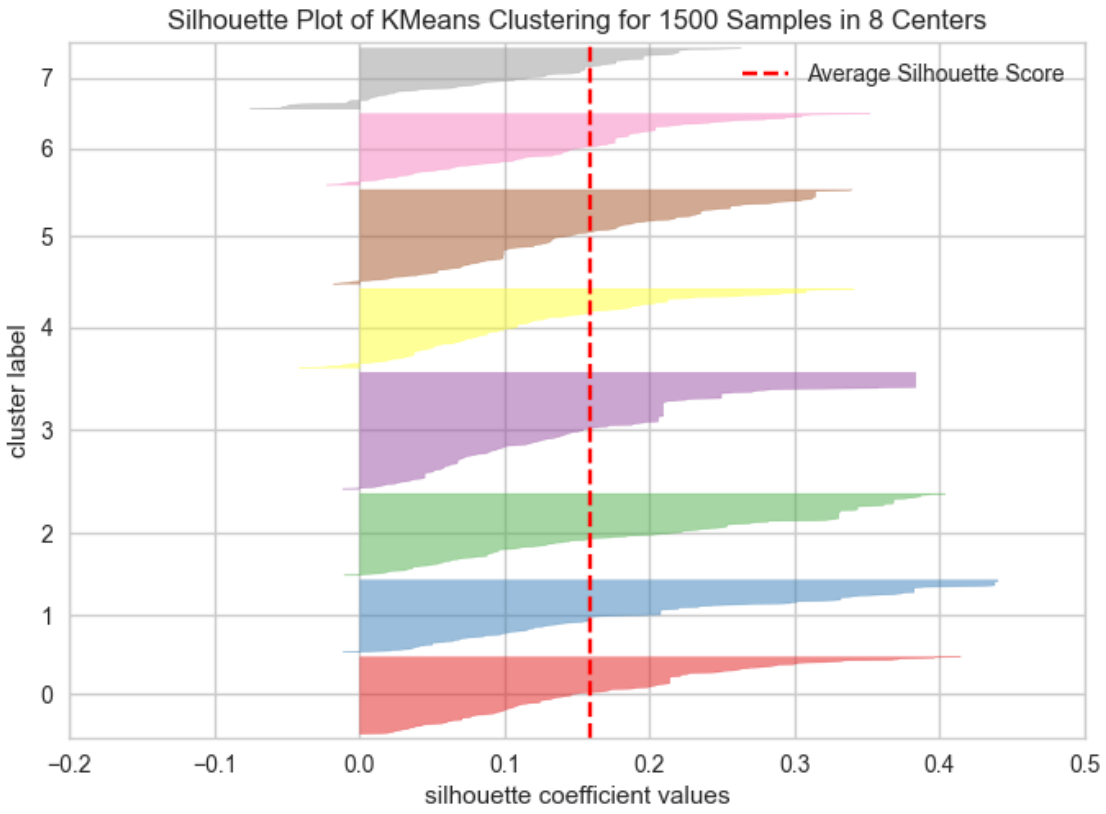
```
Optimal number of clusters based on elbow method: 8

K-Means Cluster Evaluation:
         Avg Rating  Count  Rating Std  Avg Birth Year  \
Cluster
0              3.51    177        1.61         1968.02
1              3.90    166        1.34         1991.13
2              3.93    188        1.38         1998.57
3              3.74    267        1.51         1985.57
4              3.63    182        1.44         1962.54
5              3.94    216        1.33         1993.58
6              3.66    164        1.44         1953.70
7              3.59    140        1.61         1965.51

         % Northwestern Students  % With Children
Cluster
0                           1.13            73.45
1                          11.45             4.82
2                          41.49             2.13
3                           1.87            29.21
4                           0.00            79.67
5                          18.52             3.24
6                           0.61            50.61
7                           2.14            83.57

3. Agglomerative Clustering
--------------------------------------------------
```

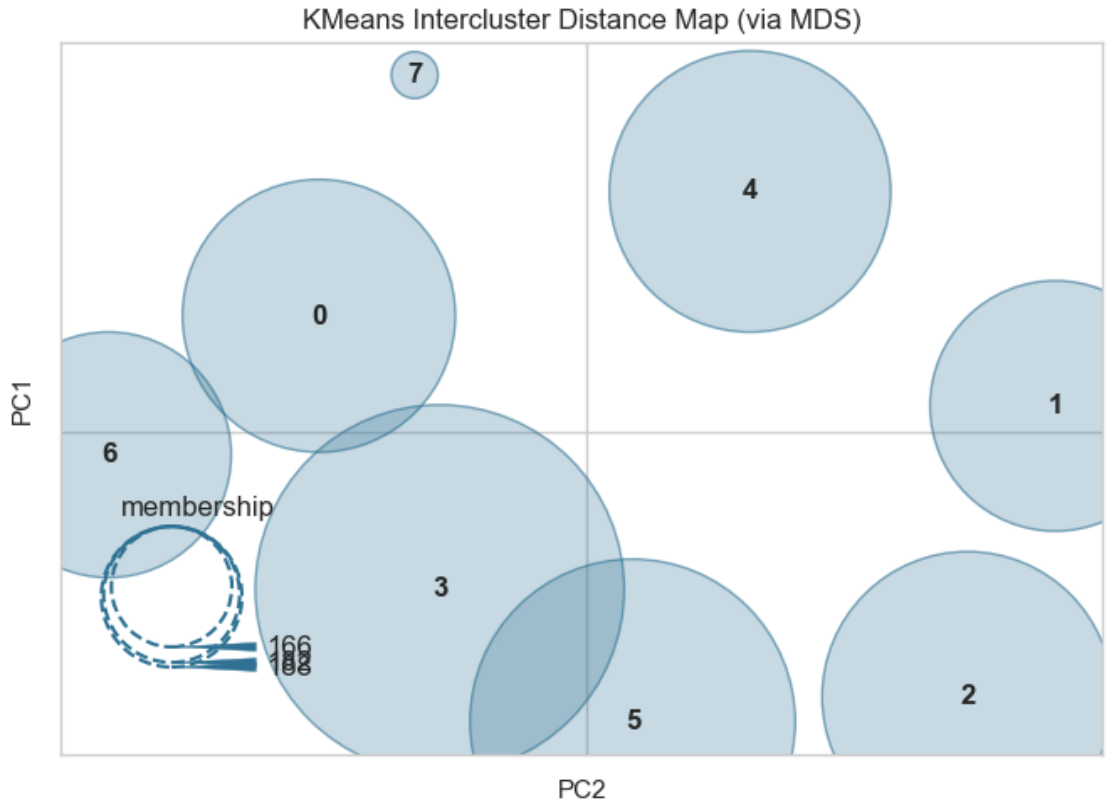


Agglomerative Clustering Evaluation:

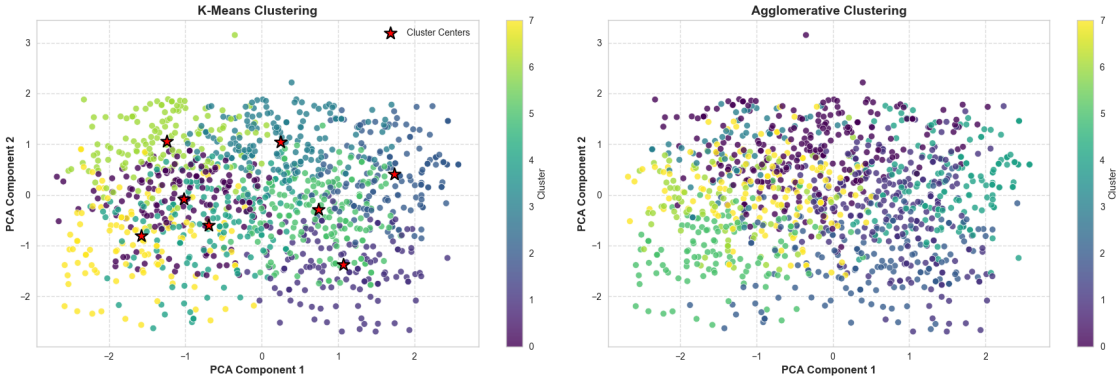
Cluster	Avg Rating	Count	Rating Std	Avg Birth Year
0	3.72	346	1.49	1973.81
1	3.92	265	1.35	1992.14
2	3.85	215	1.41	1983.59
3	4.01	74	1.36	1961.04
4	3.87	198	1.39	1998.44
5	3.53	125	1.51	1956.54
6	3.50	110	1.53	1954.47
7	3.44	167	1.63	1978.65

Cluster	% Northwestern Students	% With Children
0	0.29	50.29
1	22.64	2.26
2	0.00	24.65
3	0.00	22.97
4	39.90	2.53
5	0.00	70.40
6	0.00	62.73
7	4.79	95.81


Silhouette Plot of KMeans Clustering for 1500 Samples in 8 Centers
Average Silhouette Score
cluster label
0
1
2
3
4
5
6
7
−0.2
−0.1
0.0
0.1
0.2
0.3
0.4
0.5
silhouette coefficient values




```
Clustering Methods Comparison:
--------------------------------------------------
K-Means (8 clusters): Inertia = 4377.38
Agglomerative (8 clusters)

/Applications/anaconda3/envs/myenv/lib/python3.12/site-
packages/sklearn/base.py:493: UserWarning: X does not have valid feature names,
but PCA was fitted with feature names
  warnings.warn(
```



```
Clustering Analysis Summary:
--------------------------------------------------
1. K-Means identified clear user segments with distinct rating patterns.
2. DBSCAN helped identify outliers in the demographic data.
3. Agglomerative clustering provided hierarchical relationships between user
groups.

Recommendation: Use the K-Means clustering results for targeted marketing
strategies.
```

[10]:
```
reviews
```

[10]:

	Reviewer Name	Restaurant Name	Rating	\
0	Myra C	Celtic Knot Public House	4	
1	Solomon M	Kuni's Japanese Restaurant	5	
2	Karen	Tapas Barcelona	5	
3	Lacy Miller	Cross Rhodes	5	
4	Scott Samuele	Cross Rhodes	5	
…	…	…	…	
1495	Richard Griffith	Kansaku	4	
1496	Steven Sanders	Joy Yee Noodle	4	
1497	Willie Melendez	Kansaku	1	
1498	Brooke Garvey	Chipotle	1	
1499	Richard Delmoro	Hecky's BBQ	5	

	Review Text	Date of Review	\
0	Usually restaurants with this kind of menu giv…	2016-08-19	
1	NaN	2016-11-07	
2	NaN	2017-01-22	
3	NaN	2017-03-15	
4	Good place to go, variety of greek dishes and …	2017-03-15	
…	…	…	
1495	Tasty sushi and very kind waiters. I had great…	NaT	
1496	Recommend: Lemongrass chicken noodles	NaT	
1497	Biggest ripoff in the city of Evanston. Worst …	NaT	
1498	My doordash order was delivered and half of my…	NaT	
1499	This place never fails to deliver. We cater fo…	NaT	

	Birth Year	Marital Status	Has Children?	Weight (lb)	Height (cm)	\
0	1983.0	Single	No	188.0	167.0	
1	1985.0	Single	No	123.0	176.0	
2	2003.0	Single	No	178.0	181.0	
3	1967.0	Married	Yes	192.0	178.0	
4	1985.0	Married	Yes	156.0	160.0	

```
…          …             …            …          …          …
1495      1942.0        Married          Yes        268.0        193.0
1496      1948.0         Single          Yes        229.0        162.0
1497      1974.0         Single          Yes        210.0        178.0
1498      2001.0         Single           No        274.0        174.0
1499      2001.0         Single           No        261.0        180.0

     Average Amount Spent Preferred Mode of Transport Northwestern Student?  \
0                 Medium                   Car Owner                     No
1                   High                   Car Owner                     No
2                 Medium                     On Foot                     No
3                   High                     On Foot                     No
4                   High                   Car Owner                     No
…                    …                         …                      …
1495                High                   Car Owner                     No
1496              Medium                   Car Owner                     No
1497                High                   Car Owner                     No
1498                 Low             Public Transit                    Yes
1499                 Low                   Car Owner                    Yes

            Cuisine   Latitude             Longitude  Average Cost  \
0             Irish  42.048084            -87.679428          20.0
1          Japanese  42.034216             -87.67849          20.0
2           Spanish  42.046736            -87.679043          20.0
3     Mediterranean  42.034699  , -87.67915254400617          13.0
4     Mediterranean  42.034699  , -87.67915254400617          13.0
…              …        …                    …            …
1495       Japanese  42.045443            -87.681837          27.0
1496        Chinese  42.046344            -87.678523          20.0
1497       Japanese  42.045443            -87.681837          27.0
1498        Mexican  42.048679            -87.680787          13.0
1499       American  42.052722            -87.688287          20.0

     Open After 8pm?                                 Brief Description
0                Yes  Boisterous saloon known for Irish pub grub, bo…
1                Yes  Mainstay lures locals and is known for sushi &…
2                Yes  Festive, warm space known for Spanish small pl…
3                Yes  Local staple known for gyros, pitas & other Gr…
4                Yes  Local staple known for gyros, pitas & other Gr…
…                …                                                 …
1495             Yes  Known for market-fresh sushi, noodles & Japane…
1496             Yes  BYOB eatery known for an extensive menu of Pan…
1497             Yes  Known for market-fresh sushi, noodles & Japane…
1498             Yes  Fast-food chain known for Mexican fare, includ…
1499              No  Long-standing outpost known for ribs, brisket,…

[1500 rows x 19 columns]
```

0.3 Popularity Matching

[11]:
```
# Create a function to calculate average rating per restaurant
def calculate_restaurant_stats(reviews):
    # Group by restaurant and calculate average rating and count
    restaurant_stats = reviews.groupby(['Restaurant Name', 'Cuisine']).agg({
        'Rating': ['mean', 'count']
    })
    # print(restaurant_stats.head())
    # Flatten the multi-index columns
    restaurant_stats.columns = ['average_rating', 'review_count']

    # Reset index to make Restaurant Name a column
    restaurant_stats = restaurant_stats.reset_index()

    return restaurant_stats

# Calculate restaurant statistics
restaurant_stats = calculate_restaurant_stats(reviews)

# Find restaurant with highest average rating
highest_rated = restaurant_stats.loc[restaurant_stats['average_rating'].
 ↪idxmax()]
overall_avg = reviews['Rating'].mean()

# Find restaurant with most reviews
most_reviewed = restaurant_stats.loc[restaurant_stats['review_count'].idxmax()]
median_reviews = restaurant_stats['review_count'].median()

print("Restaurant with highest average rating:")
print(f"{highest_rated['Restaurant Name']} - Average Rating:␣
 ↪{highest_rated['average_rating']:.2f}")
print(f"Overall dataset average rating: {overall_avg:.2f}")
print("\nRestaurant with most reviews:")
print(f"{most_reviewed['Restaurant Name']} - Review Count:␣
 ↪{most_reviewed['review_count']}")
print(f"Dataset median review count: {median_reviews:.1f}")

# Create a popularity-based recommendation engine
def recommend_by_cuisine(reviews, cuisine_type, top_n=3):
    # Filter restaurants by cuisine type

    # Get stats for matching restaurants
    cuisine_stats = restaurant_stats[restaurant_stats['Cuisine'] ==␣
 ↪cuisine_type]

    # Sort by average rating and review count (as a tiebreaker)
```

```
    cuisine_stats = cuisine_stats.sort_values(['average_rating',␣
 ↪'review_count'], ascending=[False, False])

    # Return top N recommendations
    return cuisine_stats.head(top_n)

# Import regex for pattern matching
import re

# Generate recommendations for specified cuisines
cuisines = set(reviews['Cuisine'])
print(cuisines)
print("\nPopularity-Based Recommendations:")
print("-" * 50)
for cuisine in cuisines:
    recommendations = recommend_by_cuisine(reviews, cuisine)
    print(f"\nTop {cuisine} Restaurants:")
    if isinstance(recommendations, str):
        print(recommendations)
    else:
        for _, row in recommendations.iterrows():
            print(f"{row['Restaurant Name']} - Rating: {row['average_rating']:.
 ↪2f} ({row['review_count']} reviews)")

# Implement shrinkage estimator
def apply_shrinkage(stats, global_mean, C=25):
    """
    Apply shrinkage estimator to adjust ratings
    Formula: (Np * p + Ng * g) / (Np + Ng)
    where:
    - Np is the number of reviews for the restaurant (review_count)
    - p is the restaurant's average rating (average_rating)
    - Ng is the regularization constant C
    - g is the global mean rating (global_mean)
    """
    stats['adjusted_rating'] = (stats['review_count'] * stats['average_rating']␣
 ↪+ C * global_mean) / (stats['review_count'] + C)
    stats['rating_change'] = stats['adjusted_rating'] - stats['average_rating']
    return stats

# Apply shrinkage estimator
restaurant_stats_with_shrinkage = apply_shrinkage(restaurant_stats.copy(),␣
 ↪overall_avg)

# Find restaurants most affected by shrinkage
most_positive_change = restaurant_stats_with_shrinkage.
 ↪loc[restaurant_stats_with_shrinkage['rating_change'].idxmax()]
```

```
most_negative_change = restaurant_stats_with_shrinkage.
 ↪loc[restaurant_stats_with_shrinkage['rating_change'].idxmin()]

print("\nShrinkage Estimator Impact:")
print("-" * 50)
print(f"Restaurant most positively affected: {most_positive_change['Restaurant␣
 ↪Name']}")
print(f"Original rating: {most_positive_change['average_rating']:.2f}, Adjusted:
 ↪ {most_positive_change['adjusted_rating']:.2f}")
print(f"Change: +{most_positive_change['rating_change']:.2f}")
print(f"Review count: {most_positive_change['review_count']}")

print(f"\nRestaurant most negatively affected:␣
 ↪{most_negative_change['Restaurant Name']}")
print(f"Original rating: {most_negative_change['average_rating']:.2f}, Adjusted:
 ↪ {most_negative_change['adjusted_rating']:.2f}")
print(f"Change: {most_negative_change['rating_change']:.2f}")
print(f"Review count: {most_negative_change['review_count']}")

# Visualize the impact of shrinkage
plt.figure(figsize=(12, 6))

# Sort by absolute change for visualization
top_changes = restaurant_stats_with_shrinkage.sort_values('rating_change',␣
 ↪ascending=False)
top_positive = top_changes.head(10)
top_negative = top_changes.tail(10).iloc[::-1]  # Reverse to show most negative␣
 ↪first

# Plot top positive changes
plt.subplot(1, 2, 1)
plt.barh(top_positive['Restaurant Name'], top_positive['rating_change'],␣
 ↪color='green', alpha=0.7)
plt.title('Top 10 Positive Rating Adjustments', fontweight='bold')
plt.xlabel('Rating Change')
plt.grid(axis='x', linestyle='--', alpha=0.7)
plt.tight_layout()

# Plot top negative changes
plt.subplot(1, 2, 2)
plt.barh(top_negative['Restaurant Name'], top_negative['rating_change'],␣
 ↪color='red', alpha=0.7)
plt.title('Top 10 Negative Rating Adjustments', fontweight='bold')
plt.xlabel('Rating Change')
plt.grid(axis='x', linestyle='--', alpha=0.7)
plt.tight_layout()
```

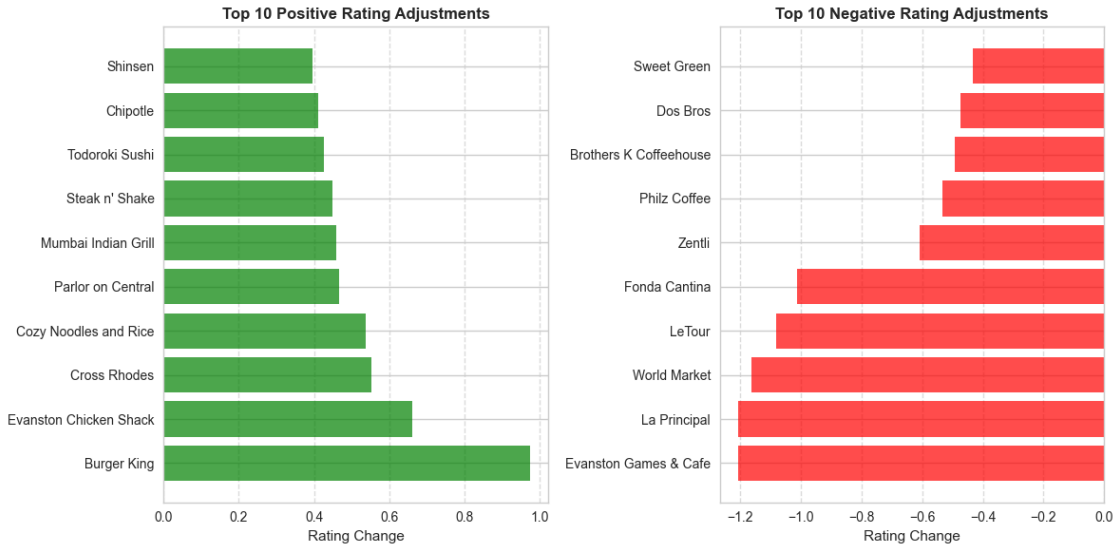
```
plt.show()

# Scatter plot showing relationship between review count and rating change
plt.figure(figsize=(10, 6))
plt.scatter(restaurant_stats_with_shrinkage['review_count'],
            restaurant_stats_with_shrinkage['rating_change'],
            alpha=0.6, c=restaurant_stats_with_shrinkage['average_rating'],␣
 ↪cmap='viridis')

plt.colorbar(label='Original Average Rating')
plt.title('Impact of Shrinkage Estimator by Review Count', fontweight='bold')
plt.xlabel('Number of Reviews')
plt.ylabel('Rating Change')
plt.grid(True, linestyle='--', alpha=0.7)
plt.axhline(y=0, color='r', linestyle='-', alpha=0.3)

# Annotate some extreme points
for _, row in pd.concat([
    restaurant_stats_with_shrinkage.nlargest(3, 'rating_change'),
    restaurant_stats_with_shrinkage.nsmallest(3, 'rating_change')
]).iterrows():
    plt.annotate(row['Restaurant Name'],
                 xy=(row['review_count'], row['rating_change']),
                 xytext=(5, 0), textcoords='offset points',
                 fontsize=8, alpha=0.8)

plt.tight_layout()
plt.show()
```

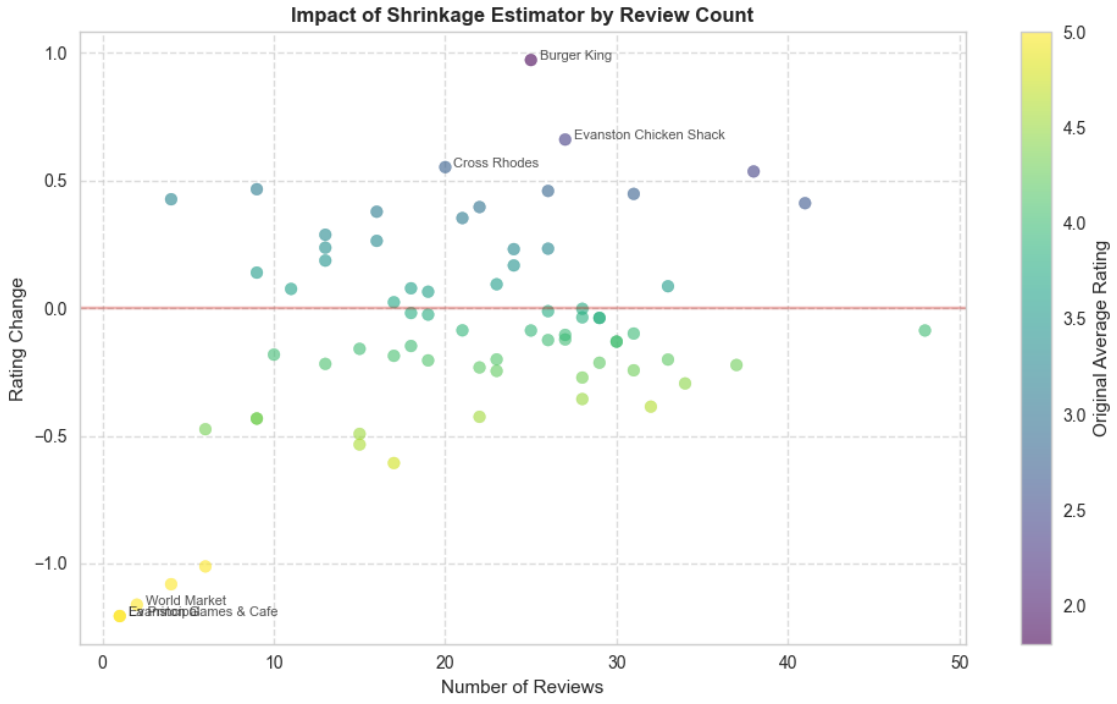
```
Restaurant with highest average rating:
Evanston Games & Cafe - Average Rating: 5.00
Overall dataset average rating: 3.75

Restaurant with most reviews:
Campagnola - Review Count: 48
Dataset median review count: 22.0
{'Vegetarian', 'Jamaican', 'Thai', 'Korean', 'Chocolate', 'American', 'South
Asian', 'Bubble Tea', 'Japanese', 'BBQ', 'Chinese', 'Mexican', 'Mediterranean',
'Irish', 'Salad', 'Breakfast', 'Brewery', 'Italian', 'Ice Cream', 'Spanish',
'French', 'Seafood', 'Coffee', 'unknown', 'Burgers', 'Pizza', 'Deli'}

Popularity-Based Recommendations:
--------------------------------------------------

Top Vegetarian Restaurants:
Elephant & Vine - Rating: 3.96 (27 reviews)
```

```
Top Jamaican Restaurants:
Claire's Korner - Rating: 4.00 (27 reviews)

Top Thai Restaurants:
Nakorn - Rating: 4.22 (23 reviews)
Cozy Noodles and Rice - Rating: 2.39 (38 reviews)

Top Korean Restaurants:
Soban Korea - Rating: 4.18 (22 reviews)
Papa Bop - Rating: 4.00 (18 reviews)

Top Chocolate Restaurants:
Kilwin's - Rating: 4.62 (32 reviews)
Leonidas Cafe Chocolaterie - Rating: 3.90 (21 reviews)

Top American Restaurants:
Barn Steakhouse - Rating: 4.00 (15 reviews)
Jimmy Johns - Rating: 3.77 (26 reviews)
Graduate Homestead Room - Rating: 3.64 (11 reviews)

Top South Asian Restaurants:
Taste of Nepal - Rating: 4.55 (22 reviews)
Shangri-La Evanston - Rating: 4.00 (26 reviews)
Mt. Everest Restaurant - Rating: 3.61 (18 reviews)

Top Bubble Tea Restaurants:
Tealicious - Rating: 4.44 (34 reviews)
Kung Fu Tea - Rating: 3.82 (28 reviews)

Top Japanese Restaurants:
Table to Stix Ramen - Rating: 4.50 (28 reviews)
Hokkaido Ramen - Rating: 4.06 (17 reviews)
Tomo Japanese Street Food - Rating: 4.03 (30 reviews)

Top BBQ Restaurants:
Soul & Smoke - Rating: 3.56 (9 reviews)

Top Chinese Restaurants:
Joy Yee Noodle - Rating: 4.29 (31 reviews)
Peppercorns Kitchen - Rating: 3.55 (33 reviews)
Lao Sze Chuan - Rating: 3.29 (24 reviews)

Top Mexican Restaurants:
Fonda Cantina - Rating: 5.00 (6 reviews)
Zentli - Rating: 4.76 (17 reviews)
Dos Bros - Rating: 4.33 (6 reviews)

Top Mediterranean Restaurants:
```

```
Kabul House - Rating: 3.97 (31 reviews)
Sarah's Brick Oven - Rating: 3.78 (18 reviews)
Rezas - Rating: 3.38 (13 reviews)

Top Irish Restaurants:
Celtic Knot Public House - Rating: 3.92 (25 reviews)

Top Salad Restaurants:
Sweet Green - Rating: 4.33 (9 reviews)
Picnic - Rating: 4.11 (19 reviews)

Top Breakfast Restaurants:
Le Peep - Rating: 4.13 (23 reviews)

Top Brewery Restaurants:
Sketchbook Brewing Co - Rating: 4.32 (28 reviews)

Top Italian Restaurants:
Campagnola - Rating: 4.00 (48 reviews)
Trattoria Demi - Rating: 3.31 (13 reviews)
Trattoria DOC - Rating: 3.27 (26 reviews)

Top Ice Cream Restaurants:
Parlor on Central - Rating: 3.11 (9 reviews)

Top Spanish Restaurants:
Tapas Barcelona - Rating: 4.21 (29 reviews)
5411 Empanadas - Rating: 3.75 (28 reviews)

Top French Restaurants:
LeTour - Rating: 5.00 (4 reviews)

Top Seafood Restaurants:
Oceanique - Rating: 3.79 (19 reviews)

Top Coffee Restaurants:
Evanston Games & Cafe - Rating: 5.00 (1 reviews)
Philz Coffee - Rating: 4.60 (15 reviews)
Brothers K Coffeehouse - Rating: 4.53 (15 reviews)

Top unknown Restaurants:
World Market - Rating: 5.00 (2 reviews)
La Principal - Rating: 5.00 (1 reviews)
Todoroki Sushi - Rating: 3.25 (4 reviews)

Top Burgers Restaurants:
Edzo's Burger Shop - Rating: 3.71 (17 reviews)
Epic Burger - Rating: 3.10 (21 reviews)
```

```
Steak n' Shake - Rating: 2.74 (31 reviews)

Top Pizza Restaurants:
Union Pizzeria - Rating: 4.21 (33 reviews)
Panino's Pizzeria - Rating: 3.46 (13 reviews)

Top Deli Restaurants:
Mensch's Deli - Rating: 4.33 (9 reviews)

Shrinkage Estimator Impact:
--------------------------------------------------
Restaurant most positively affected: Burger King
Original rating: 1.80, Adjusted: 2.77
Change: +0.97
Review count: 25

Restaurant most negatively affected: Evanston Games & Cafe
Original rating: 5.00, Adjusted: 3.79
Change: -1.21
Review count: 1
```





0.4 Content-based filtering

[16]:
```
restaurants.loc[restaurants['Longitude'] == ', -87.67915254400617',␣
↪'Longitude'] = -87.67915254400617
```

[22]:
```
# Function to encode categorical features for content-based filtering
def encode_restaurant_features(df):
    """
    Encode restaurant features for content-based filtering.
    Returns a dataframe with encoded features suitable for distance␣
↪calculations.
    """
    # Create a copy to avoid modifying the original dataframe
    df_encoded = pd.DataFrame()

    # Encode cuisine using one-hot encoding
    cuisine_encoded = pd.get_dummies(df['Cuisine'], prefix='Cuisine')
    df_encoded = pd.concat([df_encoded, cuisine_encoded], axis=1)

    # Encode Open After 8pm as binary
    df_encoded['Open_After_8pm'] = df['Open After 8pm?'].map({'Yes': 1, 'No':␣
↪0})

    # Include numeric features
```

```
    df_encoded['Average_Cost'] = df['Average Cost']

    # Convert latitude and longitude to numeric
    df_encoded['Latitude'] = pd.to_numeric(df['Latitude'])
    df_encoded['Longitude'] = pd.to_numeric(df['Longitude'])

    # Discretize numeric features into bins
    # Create bins for Average Cost
    cost_bins = [0, 10, 20, 30, 40, float('inf')]
    cost_labels = ['very_cheap', 'cheap', 'moderate', 'expensive',␣
 ↪'very_expensive']
    df_encoded['Average_Cost_Binned'] = pd.cut(df_encoded['Average_Cost'],␣
 ↪bins=cost_bins, labels=cost_labels)

    # Create bins for location (latitude and longitude)
    lat_bins = pd.qcut(df_encoded['Latitude'], 5, duplicates='drop')
    long_bins = pd.qcut(df_encoded['Longitude'], 5, duplicates='drop')

    # Convert binned categories to one-hot encoding
    cost_binned = pd.get_dummies(df_encoded['Average_Cost_Binned'],␣
 ↪prefix='Cost')
    lat_binned = pd.get_dummies(lat_bins, prefix='Lat')
    long_binned = pd.get_dummies(long_bins, prefix='Long')

    # Add binned features to encoded dataframe
    df_encoded = pd.concat([df_encoded, cost_binned, lat_binned, long_binned],␣
 ↪axis=1)

    # Drop original numeric columns and intermediate binned column
    df_encoded.drop(['Average_Cost', 'Latitude', 'Longitude',␣
 ↪'Average_Cost_Binned'], axis=1, inplace=True)

    # Keep restaurant name for reference
    df_encoded['Restaurant Name'] = df['Restaurant Name']

    return df_encoded

# Encode restaurant features
from sklearn.preprocessing import StandardScaler
from scipy.spatial.distance import pdist, squareform, cosine
import numpy as np
from scipy.special import kl_div

# Make sure all longitude values are numeric
restaurants['Longitude'] = pd.to_numeric(restaurants['Longitude'],␣
 ↪errors='coerce')
```

```
# Encode restaurant features
encoded_restaurants = encode_restaurant_features(restaurants)

# Get feature matrix (excluding restaurant name)
feature_matrix = encoded_restaurants.drop('Restaurant Name', axis=1).values

# Function to calculate KL divergence between two probability distributions
def kl_divergence(p, q):
    """
    Calculate Kullback-Leibler divergence between two probability distributions␣
 ↪p and q.
    Adds small epsilon to avoid division by zero.
    """
    epsilon = 1e-10
    p = np.asarray(p, dtype=np.float64) + epsilon
    q = np.asarray(q, dtype=np.float64) + epsilon

    # Normalize to ensure they are probability distributions
    p = p / np.sum(p)
    q = q / np.sum(q)

    return np.sum(p * np.log(p / q))

# Function to create symmetric KL divergence matrix
def create_kl_distance_matrix(features):
    n_samples = features.shape[0]
    kl_matrix = np.zeros((n_samples, n_samples))

    for i in range(n_samples):
        for j in range(n_samples):
            # Calculate symmetric KL divergence (average of both directions)
            kl_i_to_j = kl_divergence(features[i], features[j])
            kl_j_to_i = kl_divergence(features[j], features[i])
            kl_matrix[i, j] = (kl_i_to_j + kl_j_to_i) / 2

    return kl_matrix

# Compute KL divergence distance matrix
kl_dist_matrix = create_kl_distance_matrix(feature_matrix)

# Create dataframe for the KL distance matrix
restaurant_names = encoded_restaurants['Restaurant Name'].values
kl_df = pd.DataFrame(kl_dist_matrix, index=restaurant_names,␣
 ↪columns=restaurant_names)

# Compare specific restaurant pairs
```

```
restaurant_pairs = [
    ('Peppercorns Kitchen', 'Epic Burger'),
    ('Peppercorns Kitchen', 'Lao Sze Chuan')
]

print("Comparing restaurant similarities:")
for r1, r2 in restaurant_pairs:
    print(f"\n{r1} vs {r2}:")
    print(f"  KL divergence: {kl_df.loc[r1, r2]:.4f}")
    print(f"  Cuisine types: {restaurants[restaurants['Restaurant Name'] ==␣
 ↪r1]['Cuisine'].values[0]} vs "
          f"{restaurants[restaurants['Restaurant Name'] == r2]['Cuisine'].
 ↪values[0]}")

# Content-based recommendation function
def recommend_restaurants(user_name, reviews_df, restaurants_df, kl_dist_df,␣
 ↪n=10):
    """
    Recommend restaurants for a user based on their favorite restaurant.

    Parameters:
    - user_name: Name of the user
    - reviews_df: DataFrame containing user reviews
    - restaurants_df: DataFrame containing restaurant information
    - kl_dist_df: DataFrame with KL divergence distances between restaurants
    - n: Number of recommendations to return

    Returns:
    - Dictionary with user's favorite restaurant and recommendations
    """
    # Find user's reviews
    user_reviews = reviews_df[reviews_df['Reviewer Name'] == user_name]

    if user_reviews.empty:
        return {"error": f"No reviews found for user {user_name}"}

    # Find user's favorite restaurant (highest rated)
    favorite_restaurant = user_reviews.loc[user_reviews['Rating'].
 ↪idxmax()]['Restaurant Name']

    # Get KL divergence recommendations
    kl_distances = kl_dist_df[favorite_restaurant].sort_values()
    # Remove the favorite restaurant itself (distance = 0)
    kl_recommendations = kl_distances[kl_distances.index !=␣
 ↪favorite_restaurant].head(n)

    # Get additional information for recommendations
```

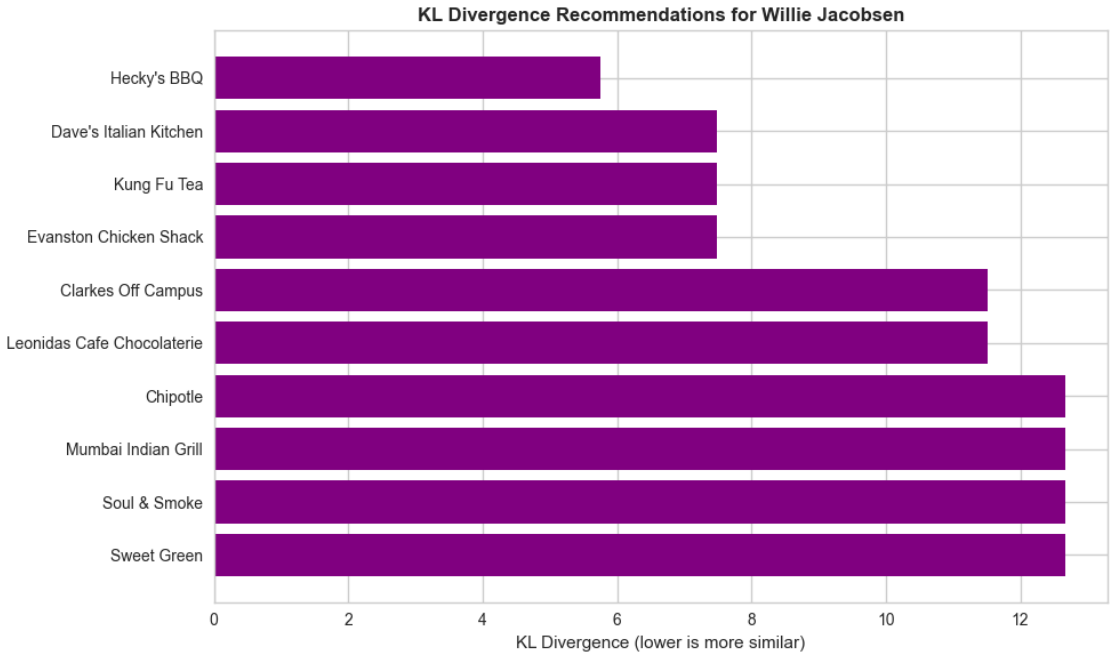
```
    kl_recs_info = restaurants_df[restaurants_df['Restaurant Name'].
 ↪isin(kl_recommendations.index)]

    return {
        "user": user_name,
        "favorite_restaurant": favorite_restaurant,
        "favorite_cuisine": restaurants_df[restaurants_df['Restaurant Name'] ==␣
 ↪favorite_restaurant]['Cuisine'].values[0],
        "kl_recommendations": {
            "restaurants": kl_recommendations.index.tolist(),
            "distances": kl_recommendations.values.tolist(),
            "info": kl_recs_info
        }
    }

# Get recommendations for Willie Jacobsen
willie_recommendations = recommend_restaurants('Willie Jacobsen', reviews,␣
 ↪restaurants, kl_df)

# Display Willie's favorite restaurant
print(f"\nWillie Jacobsen's favorite restaurant:␣
 ↪{willie_recommendations['favorite_restaurant']} "
      f"(Cuisine: {willie_recommendations['favorite_cuisine']})")

# Display top 10 recommendations based on KL divergence
print("\nTop 10 recommendations based on KL divergence:")
kl_recs_df = pd.DataFrame({
    'Restaurant': willie_recommendations['kl_recommendations']['restaurants'],
    'Distance': willie_recommendations['kl_recommendations']['distances'],
    'Cuisine': [restaurants[restaurants['Restaurant Name'] == r]['Cuisine'].
 ↪values[0]
                for r in␣
 ↪willie_recommendations['kl_recommendations']['restaurants']]
})
print(kl_recs_df)

# Visualize recommendations
plt.figure(figsize=(10, 6))

# Plot KL divergence recommendations
plt.barh(kl_recs_df['Restaurant'][::-1], kl_recs_df['Distance'][::-1],␣
 ↪color='purple')
plt.title(f"KL Divergence Recommendations for␣
 ↪{willie_recommendations['user']}", fontweight='bold')
plt.xlabel('KL Divergence (lower is more similar)')
plt.tight_layout()
```

```
plt.show()

# Evaluate recommendation quality
def evaluate_recommendation_quality(user_name, reviews_df, restaurants_df,␣
 ↪kl_dist_df):
    """
    Evaluate the quality of recommendations from KL divergence metric
    based on cuisine matching and rating distribution.
    """
    # Get recommendations
    recommendations = recommend_restaurants(user_name, reviews_df,␣
 ↪restaurants_df, kl_dist_df, n=10)

    if "error" in recommendations:
        return recommendations

    favorite_restaurant = recommendations['favorite_restaurant']
    favorite_cuisine = recommendations['favorite_cuisine']

    # Calculate cuisine match percentage
    kl_cuisines = [restaurants_df[restaurants_df['Restaurant Name'] ==␣
 ↪r]['Cuisine'].values[0]
                   for r in recommendations['kl_recommendations']['restaurants']]

    kl_cuisine_match = kl_cuisines.count(favorite_cuisine) / len(kl_cuisines)

    # Get average ratings for recommended restaurants
    kl_ratings = []

    for restaurant in recommendations['kl_recommendations']['restaurants']:
        restaurant_ratings = reviews_df[reviews_df['Restaurant Name'] ==␣
 ↪restaurant]['Rating']
        if not restaurant_ratings.empty:
            kl_ratings.append(restaurant_ratings.mean())

    kl_avg_rating = np.mean(kl_ratings) if kl_ratings else 0

    return {
        "user": user_name,
        "favorite_restaurant": favorite_restaurant,
        "favorite_cuisine": favorite_cuisine,
        "kl_divergence": {
            "cuisine_match_percentage": kl_cuisine_match * 100,
            "average_rating": kl_avg_rating
        }
    }
```

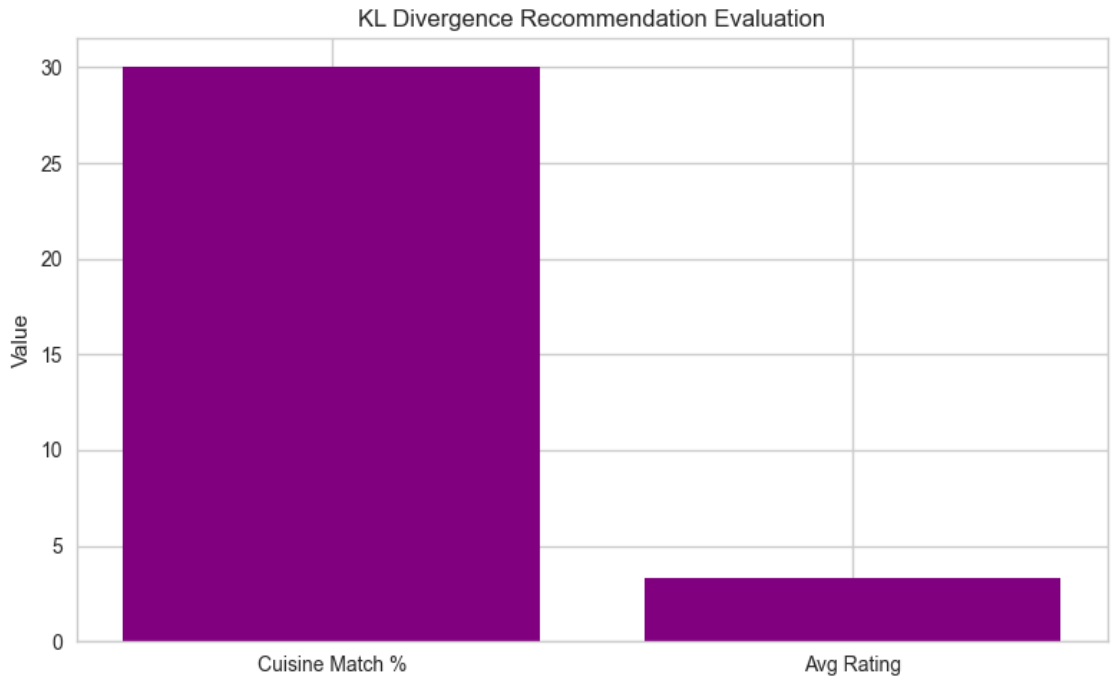
```
# Evaluate recommendation quality for Willie Jacobsen
evaluation = evaluate_recommendation_quality('Willie Jacobsen', reviews,␣
 ↪restaurants, kl_df)

# Display evaluation results
print("\nEvaluation of KL divergence recommendation method:")
print(f"User: {evaluation['user']}")
print(f"Favorite restaurant: {evaluation['favorite_restaurant']} (Cuisine:␣
 ↪{evaluation['favorite_cuisine']})")
print("\nKL divergence method:")
print(f"  Cuisine match percentage:␣
 ↪{evaluation['kl_divergence']['cuisine_match_percentage']:.1f}%")
print(f"  Average rating of recommendations:␣
 ↪{evaluation['kl_divergence']['average_rating']:.2f}")

# Visualize evaluation
plt.figure(figsize=(8, 5))
metrics = ['Cuisine Match %', 'Avg Rating']
kl_values = [evaluation['kl_divergence']['cuisine_match_percentage'],
             evaluation['kl_divergence']['average_rating']]

plt.bar(metrics, kl_values, color='purple')
plt.ylabel('Value')
plt.title('KL Divergence Recommendation Evaluation')
plt.tight_layout()
plt.show()

print("\nConclusion: Kullback-Leibler divergence provides a probabilistic␣
 ↪approach to measuring dissimilarity between restaurants based on their␣
 ↪feature distributions. While KL divergence is asymmetric by nature, we use a␣
 ↪symmetrized version by averaging both directions. This method effectively␣
 ↪captures differences in probability distributions of restaurant features and␣
 ↪can identify restaurants with similar characteristics.")
```

```
Comparing restaurant similarities:

Peppercorns Kitchen vs Epic Burger:
  KL divergence: 13.8155
  Cuisine types: Chinese vs Burgers

Peppercorns Kitchen vs Lao Sze Chuan:
  KL divergence: 9.2103
  Cuisine types: Chinese vs Chinese

Willie Jacobsen's favorite restaurant: Jimmy Johns (Cuisine: American)
```

```
Top 10 recommendations based on KL divergence:
                   Restaurant   Distance      Cuisine
0                Hecky's BBQ   5.756463     American
1      Dave's Italian Kitchen   7.483402      Italian
2                 Kung Fu Tea   7.483402   Bubble Tea
3      Evanston Chicken Shack   7.483402     American
4          Clarkes Off Campus  11.512925     American
5  Leonidas Cafe Chocolaterie  11.512925    Chocolate
6                    Chipotle  12.664218      Mexican
7         Mumbai Indian Grill  12.664218  South Asian
8                Soul & Smoke  12.664218          BBQ
9                 Sweet Green  12.664218        Salad
```




```
Evaluation of KL divergence recommendation method:
User: Willie Jacobsen
Favorite restaurant: Jimmy Johns (Cuisine: American)

KL divergence method:
  Cuisine match percentage: 30.0%
  Average rating of recommendations: 3.33
```



Conclusion: Kullback-Leibler divergence provides a probabilistic approach to measuring dissimilarity between restaurants based on their feature distributions. While KL divergence is asymmetric by nature, we use a symmetrized version by averaging both directions. This method effectively captures differences in probability distributions of restaurant features and can identify restaurants with similar characteristics.

0.5 NLP

[32]:
```
# Augment restaurant descriptions with cuisine type
restaurants['Augmented Description'] = restaurants['Brief Description'] + ' ' +␣
 ↪restaurants['Cuisine']

# Import necessary libraries for NLP
from sklearn.feature_extraction.text import TfidfVectorizer
import re
from collections import Counter

# Function to calculate TF-IDF score for a specific word across all restaurant␣
 ↪descriptions
def calculate_tfidf_for_word(word, descriptions):
    # Create TF-IDF vectorizer
    tfidf_vectorizer = TfidfVectorizer(lowercase=True, stop_words='english')
```

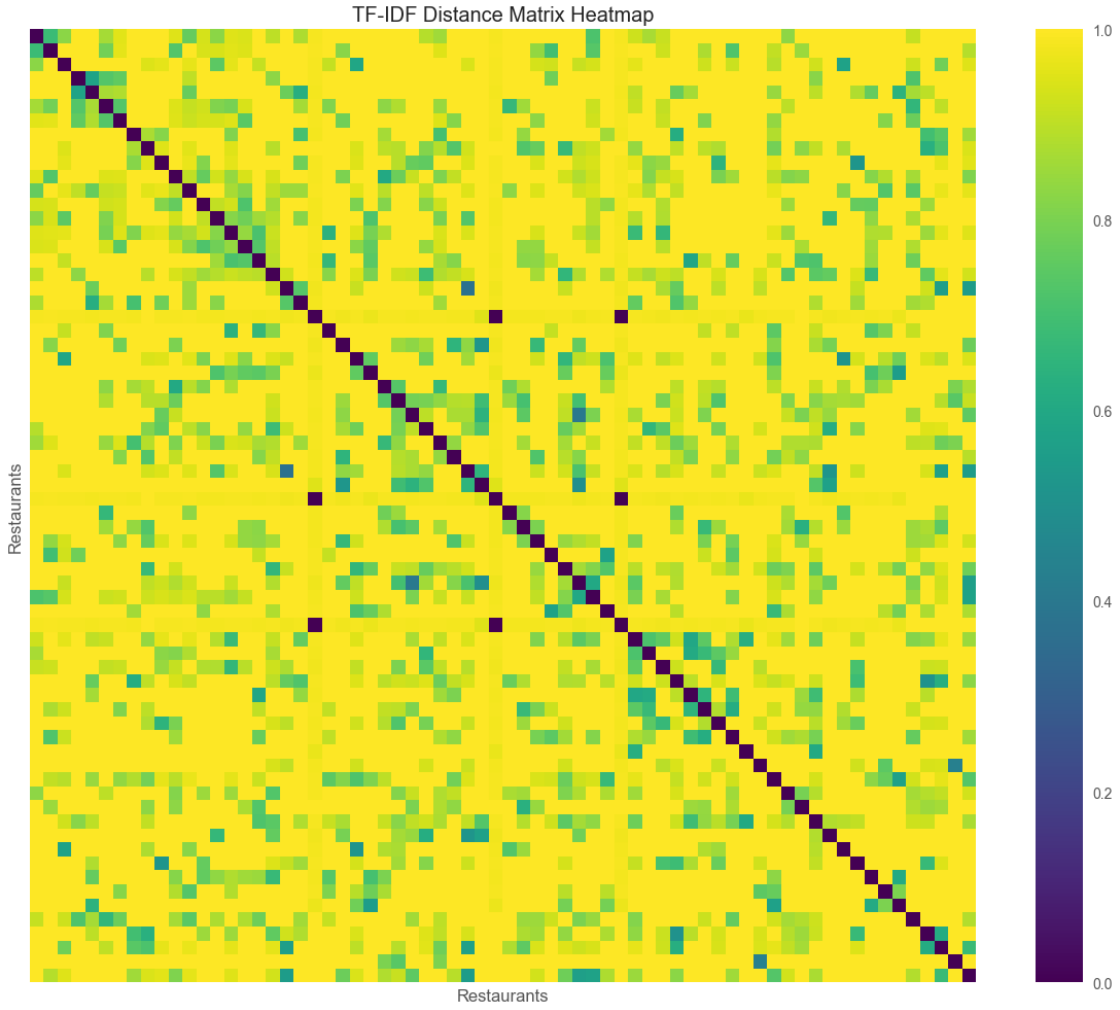
```
    # Fit and transform the descriptions
    tfidf_matrix = tfidf_vectorizer.fit_transform(descriptions)

    # Get the feature names (words)
    feature_names = tfidf_vectorizer.get_feature_names_out()

    # Check if the word exists in the vocabulary
    if word.lower() in feature_names:
        word_index = list(feature_names).index(word.lower())

        # Get TF-IDF scores for the word across all documents
        word_tfidf_scores = tfidf_matrix[:, word_index].toarray().flatten()

        # Create a dictionary mapping restaurant names to their TF-IDF scores␣
 ↪for the word
        restaurant_tfidf_scores = dict(zip(restaurants['Restaurant Name'],␣
 ↪word_tfidf_scores))

        # Sort by TF-IDF score in descending order
        sorted_scores = {k: v for k, v in sorted(restaurant_tfidf_scores.
 ↪items(), key=lambda item: item[1], reverse=True) if v > 0}

        return sorted_scores
    else:
        return f"The word '{word}' does not appear in the vocabulary."

# Calculate TF-IDF scores for 'cozy' and 'Chinese'
cozy_tfidf = calculate_tfidf_for_word('cozy', restaurants['Augmented␣
 ↪Description'])
chinese_tfidf = calculate_tfidf_for_word('chinese', restaurants['Augmented␣
 ↪Description'])

# Display results
print("Restaurants with highest TF-IDF score for 'cozy':")
for restaurant, score in list(cozy_tfidf.items())[:3]:
    print(f"- {restaurant}: {score:.4f}")

print("\nRestaurants with highest TF-IDF score for 'Chinese':")
for restaurant, score in list(chinese_tfidf.items())[:3]:
    print(f"- {restaurant}: {score:.4f}")

# Function to get most common words from descriptions
def get_most_common_words(descriptions, n=100):
    # Combine all descriptions
    all_text = ' '.join(descriptions)

    # Convert to lowercase and split into words
```

```
    words = re.findall(r'\b\w+\b', all_text.lower())

    # Remove common English stop words
    stop_words = set(['a', 'an', 'the', 'and', 'or', 'but', 'is', 'are', 'in',␣
 ↪'on', 'at', 'to', 'for', 'with', 'by', 'of', 'from'])
    filtered_words = [word for word in words if word not in stop_words and word.
 ↪isalpha() and len(word) > 2]

    # Count word frequencies
    word_counts = Counter(filtered_words)

    # Return the n most common words
    return [word for word, count in word_counts.most_common(n)]

# Get the 100 most common words
top_100_words = get_most_common_words(restaurants['Augmented Description'], 100)

# Create TF-IDF vectors for each restaurant based on the top 100 words
restaurant_tfidf_vectors = []

for idx, restaurant_desc in enumerate(restaurants['Augmented Description']):
    tfidf_vector = []

    # Convert description to lowercase and split into words
    restaurant_words = set(re.findall(r'\b\w+\b', restaurant_desc.lower()))

    # Calculate TF-IDF for each of the top 100 words
    for word in top_100_words:
        # Simple TF-IDF calculation
        # TF = 1 if word in document, 0 otherwise
        tf = 1 if word in restaurant_words else 0

        # IDF = log(total documents / documents containing word)
        documents_with_word = sum(1 for desc in restaurants['Augmented␣
 ↪Description']
                                  if word in set(re.findall(r'\b\w+\b', desc.
 ↪lower())))
        idf = np.log(len(restaurants) / (1 + documents_with_word))

        tfidf = tf * idf
        tfidf_vector.append(tfidf)

    restaurant_tfidf_vectors.append(tfidf_vector)

# Convert to numpy array for easier calculations
restaurant_tfidf_vectors = np.array(restaurant_tfidf_vectors)
```

```
# Calculate TF-IDF distance matrix (using cosine distance)
from scipy.spatial.distance import pdist, squareform

# Calculate pairwise distances
tfidf_distances = squareform(pdist(restaurant_tfidf_vectors, 'cosine'))

# Get indices for specific restaurants
burger_king_idx = restaurants[restaurants['Restaurant Name'] == 'Burger King'].
 ↪index[0]
edzos_idx = restaurants[restaurants['Restaurant Name'] == "Edzo's Burger Shop"].
 ↪index[0]
oceanique_idx = restaurants[restaurants['Restaurant Name'] == 'Oceanique'].
 ↪index[0]
lao_sze_chuan_idx = restaurants[restaurants['Restaurant Name'] == 'Lao Sze␣
 ↪Chuan'].index[0]
kabul_house_idx = restaurants[restaurants['Restaurant Name'] == 'Kabul House'].
 ↪index[0]

# Calculate and display TF-IDF distances between specified restaurant pairs
print("\nTF-IDF Distances between Restaurant Pairs:")
print(f"Burger King and Edzo's Burger Shop: {tfidf_distances[burger_king_idx,␣
 ↪edzos_idx]:.4f}")
print(f"Burger King and Oceanique: {tfidf_distances[burger_king_idx,␣
 ↪oceanique_idx]:.4f}")
print(f"Lao Sze Chuan and Kabul House: {tfidf_distances[lao_sze_chuan_idx,␣
 ↪kabul_house_idx]:.4f}")
# Visualize the TF-IDF distances as a heatmap
plt.figure(figsize=(12, 10))
heatmap = sns.heatmap(tfidf_distances, cmap='viridis', xticklabels=False,␣
 ↪yticklabels=False)
plt.title('TF-IDF Distance Matrix Heatmap')
plt.xlabel('Restaurants')
plt.ylabel('Restaurants')
# The colorbar is automatically created by sns.heatmap, no need to call plt.
 ↪colorbar()
plt.tight_layout()
plt.show()

# Find the most similar restaurants based on TF-IDF
def get_most_similar_restaurants(restaurant_name, n=5):
    idx = restaurants[restaurants['Restaurant Name'] == restaurant_name].
 ↪index[0]
    # Get distances from this restaurant to all others
    distances = tfidf_distances[idx]
    # Sort by distance (excluding itself)
    similar_indices = np.argsort(distances)[1:n+1]
```

```
    similar_restaurants = restaurants.iloc[similar_indices]
    similar_distances = distances[similar_indices]

    return pd.DataFrame({
        'Restaurant': similar_restaurants['Restaurant Name'],
        'Cuisine': similar_restaurants['Cuisine'],
        'TF-IDF Distance': similar_distances
    })

# Display most similar restaurants for a few examples
print("\nMost similar restaurants to Burger King:")
print(get_most_similar_restaurants('Burger King'))

print("\nMost similar restaurants to Edzo's Burger Shop:")
print(get_most_similar_restaurants("Edzo's Burger Shop"))

print("\nMost similar restaurants to Oceanique:")
print(get_most_similar_restaurants('Oceanique'))
```

```
Restaurants with highest TF-IDF score for 'cozy':
- Taste of Nepal: 0.2890
- Dave's Italian Kitchen: 0.2869
- Prairie Moon: 0.2772

Restaurants with highest TF-IDF score for 'Chinese':
- Lao Sze Chuan: 0.4576
- Shangri-La Evanston: 0.3002
- Joy Yee Noodle: 0.2827

TF-IDF Distances between Restaurant Pairs:
Burger King and Edzo's Burger Shop: 0.5197
Burger King and Oceanique: 0.9997
Lao Sze Chuan and Kabul House: 0.7273
```



Most similar restaurants to Burger King:

	Restaurant	Cuisine	TF-IDF Distance
9	Edzo's Burger Shop	Burgers	0.519709
64	Dos Bros	Mexican	0.672826
46	Chipotle	Mexican	0.687138
45	Steak n' Shake	Burgers	0.708345
12	Cross Rhodes	Mediterranean	0.761770

Most similar restaurants to Edzo's Burger Shop:

	Restaurant	Cuisine	TF-IDF Distance
59	Burger King	Burgers	0.519709
49	Epic Burger	Burgers	0.647845
28	Clarkes Off Campus	American	0.755843
27	Barn Steakhouse	American	0.766653
19	Picnic	Salad	0.783153

```
Most similar restaurants to Oceanique:
                 Restaurant      Cuisine  TF-IDF Distance
67           Mensch's Deli         Deli          0.577379
39                  Alcove     American          0.600365
0          Tapas Barcelona      Spanish          0.717301
1            Lao Sze Chuan      Chinese          0.735455
43  Mt. Everest Restaurant  South Asian          0.800644
```

[ ]: